

Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Навчально–науковий інститут комп’ютерних систем
Кафедра інформаційних систем

КУРСОВА РОБОТА

з дисципліни «Об’єктно–орієнтоване програмування»

BACKEND–СЕРВІС УПРАВЛІННЯ ОНЛАЙН–КУРСАМИ

Студентки 2 курсу AI-243 групи
Спеціальності F3 – «Комп’ютерні науки»

Антонової К.В.
(прізвище та ініціали)

Керівник: доцент, кандидат технічних наук, Годовиченко М.А.

(посада, вчене звання, науковий ступінь, прізвище та ініціали)

Національна шкала

Кількість балів: _____

Оцінка: ECTS _____

Члени комісії

(підпис) _____ (прізвище та ініціали)

(підпис) _____ (прізвище та ініціали)

(підпис) _____ (прізвище та ініціали)

Одеса – 2026

ЗМІСТ

Вступ.....	4
1 Аналіз предметної області.....	6
1.1 Опис предметної області онлайн–курсів.....	6
1.2 Аналіз вимог до backend–сервісу.....	7
1.3 Опис основних користувачів системи.....	8
1.4 Постановка задачі курсової роботи.....	9
Висновки до розділу 1.....	10
2 Проєктування backend–сервісу управління онлайн–курсами.....	12
2.1 Загальна архітектура програмного забезпечення.....	12
2.2 Проєктування сутностей системи.....	13
2.3 Проєктування зв’язків між сутностями.....	14
2.4 Проєктування структури бази даних.....	14
2.5 Проєктування REST API.....	16
2.6 Проєктування повнотекстового пошуку.....	17
2.7 Проєктування аналітичних запитів.....	18
Висновки до розділу 2.....	18
3 Реалізація програмного продукту.....	20
3.1 Використані технології та інструменти.....	20
3.2 Реалізація сутностей instructor, course, student, enrollment та module.....	20
3.3 Реалізація репозиторіїв.....	21
3.4 Реалізація сервісного шару.....	22
3.5 Реалізація rest–контролерів.....	23
3.6 Реалізація DTO та валідації даних.....	24
3.7 Реалізація зв’язків між курсами, студентами, викладачами та модулями.....	25
3.8 Реалізація пошуку курсів і студентів.....	25
3.9 Реалізація аналітичних endpoint’ів.....	26
3.10 Налаштування actuator та метрик сервісу.....	26
3.11 Обробка помилок у системі.....	27

Висновки до розділу 3.....	28
4 Тестування та налагодження backend–сервісу.....	29
4.1 Тестування API викладачів.....	29
4.2 Тестування API курсів.....	31
4.3 Тестування API студентів.....	34
4.4 Тестування запису студентів на курси.....	37
4.5 Тестування API навчальних модулів.....	39
4.6 Тестування пошуку та аналітичних запитів.....	42
4.7 Перевірка працездатності сервісу через actuator.....	45
Висновки до розділу 4.....	47
Загальні висновки.....	48
Список використаних джерел.....	49
Додаток А. Вихідний код сутностей системи.....	50
Додаток Б. Вихідний код dto–класів.....	57
Додаток В. Вихідний код репозиторіїв.....	67
Додаток Г. Вихідний код сервісів.....	69
Додаток Д. Вихідний код контролерів.....	78
Додаток Е. Конфігураційні файли проєкту.....	97

ВСТУП

Сьогодні інформаційні технології відіграють важливу роль в освітньому процесі. Онлайн-навчання стало одним із поширених способів отримання знань, оскільки воно забезпечує дистанційний доступ до навчальних матеріалів, гнучкість навчання та можливість організації взаємодії між викладачами і студентами. Для ефективного функціонування таких освітніх процесів необхідні програмні системи, які дають змогу керувати курсами, студентами, викладачами, навчальними модулями та відстежувати результати навчання. Саме тому розробка backend-сервісу для управління онлайн-курсами є актуальною задачею.

Предметною областю даної курсової роботи є система управління онлайн-курсами. Така система призначена для зберігання, обробки та надання інформації про курси, викладачів, студентів, навчальні модулі та записи студентів на курси. Вона повинна забезпечувати створення нових записів, перегляд інформації, редагування та видалення даних. Також система має надавати можливість запису студентів на курси, контролю їхнього прогресу навчання, пошуку необхідної інформації та отримання статистичних даних.

Метою курсової роботи є розробка backend-сервісу для управління онлайн-курсами з використанням сучасних технологій серверної розробки.

Для досягнення поставленої мети необхідно виконати такі завдання:

- провести аналіз предметної області онлайн-курсів;
- визначити основні вимоги до backend-сервісу;
- спроектувати структуру бази даних та зв'язки між сутностями;
- реалізувати сутності викладачів, студентів, курсів, навчальних модулів та записів на курси;
- розробити REST API для взаємодії з системою;
- реалізувати операції створення, перегляду, оновлення та видалення даних;
- забезпечити можливість запису студентів на курси та обліку їхнього прогресу;

- реалізувати пошук курсів і студентів;
- реалізувати аналітичні endpoint'и для отримання статистичної інформації;
- провести тестування роботи backend-сервісу.

Для реалізації програмного продукту було обрано мову програмування Java, яка широко використовується для створення серверних застосунків [1]. Основою backend-сервісу є фреймворк Spring Boot, що спрощує створення та запуск вебзастосунків, а також дозволяє швидко реалізовувати REST API [2]–[4]. Для роботи з базою даних використано Spring Data JPA та Hibernate ORM, які забезпечують зручну взаємодію між Java-класами та реляційною базою даних [6], [7]. Як систему керування базами даних використано PostgreSQL [8]–[10]. Для документування REST API застосовано OpenAPI та SpringDoc OpenAPI [11], [12].

Результатом виконання курсової роботи є backend-сервіс управління онлайн-курсами, який дозволяє працювати з інформацією про викладачів, студентів, курси, навчальні модулі та записи студентів на курси. Розроблений сервіс забезпечує виконання основних CRUD-операцій, пошук даних, облік навчального прогресу, отримання аналітичної інформації та перевірку працездатності системи.

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ

1.1 Опис предметної області онлайн–курсів

Онлайн–курси є поширеним напрямом сучасної освіти, оскільки дають змогу організувати навчання у дистанційному форматі. Студенти можуть отримувати доступ до матеріалів, проходити курси, вивчати модулі та контролювати власний прогрес без обов’язкової фізичної присутності в навчальному закладі.

Предметною областю даної курсової роботи є система управління онлайн–курсами. Вона призначена для збереження та обробки інформації про викладачів, курси, студентів, навчальні модулі та записи студентів на курси. Така система може використовуватися як основа освітньої платформи, де викладачі створюють курси, а студенти записуються на них і проходять навчання.

Основними об’єктами предметної області є викладач, курс, студент, запис на курс та навчальний модуль. Викладач відповідає за проведення курсу. Курс містить назву, опис, рівень складності та дату створення. Навчальний модуль є частиною курсу і поділяє матеріал на окремі логічні блоки. Студент може записуватися на один або декілька курсів. Для фіксації цього використовується сутність Enrollment, яка зберігає дату запису, дату завершення та прогрес навчання.

До основних бізнес–процесів системи належать створення і редагування курсів, додавання викладачів, реєстрація студентів, запис студентів на курси, створення модулів, перегляд інформації, пошук курсів або студентів, а також отримання аналітичної інформації про навчальний процес.

Backend–сервіс у цій предметній області виконує роль серверної частини програмного забезпечення. Він приймає запити від клієнтської частини або інструментів тестування API, обробляє їх, виконує бізнес–логіку та взаємодіє з базою даних. Для реалізації такого сервісу доцільно використовувати Java, оскільки ця мова підтримує об’єктно–орієнтований підхід і широко застосовується для створення серверних застосунків [1].

1.2 Аналіз вимог до backend–сервісу

Backend–сервіс управління онлайн–курсами повинен забезпечувати основні операції над даними предметної області: створення, перегляд, оновлення та видалення інформації про викладачів, курси, студентів, записи на курси та навчальні модулі. Такі операції відповідають базовому підходу CRUD, який використовується у більшості інформаційних систем.

Функціональні вимоги включають створення нових викладачів, студентів, курсів і модулів; отримання списку записів; пошук запису за ідентифікатором; оновлення та видалення даних; додавання студента до курсу; додавання модуля до курсу; перегляд прогресу навчання; пошук курсів і студентів; отримання аналітичної інформації через окремі endpoint'и.

Окремою вимогою є реалізація зв'язків між сутностями. Один викладач може вести багато курсів, але кожен курс має одного викладача. Один курс може містити багато навчальних модулів, а кожен модуль належить одному курсу. Один студент може бути записаний на багато курсів, і один курс може мати багато студентів. Такий зв'язок реалізується через проміжну сутність Enrollment, яка зберігає інформацію про запис студента на конкретний курс.

До нефункціональних вимог належать стабільність роботи сервісу, зрозуміла структура коду, коректна обробка помилок, валідація вхідних даних, підтримка документації API та можливість перевірки працездатності сервісу. Для побудови backend–застосунку доцільно використовувати Spring Boot, оскільки він спрощує створення Java–застосунків завдяки автоматичній конфігурації та готовим засобам для розробки вебсервісів [2], [3], [4].

Для побудови багаторівневої архітектури використовується Spring Framework. Він забезпечує роботу з компонентами, впровадження залежностей та взаємодію між частинами програми [5]. Для роботи з базою даних застосовується Spring Data JPA, який спрощує створення репозиторіїв і виконання операцій доступу до даних [6]. Для об'єктно–реляційного відображення сутностей

використовується Hibernate ORM, що дозволяє пов'язувати Java-класи з таблицями реляційної бази даних [7].

Як система керування базами даних у роботі використовується PostgreSQL. Вона підходить для збереження структурованих даних, підтримує реляційну модель і роботу з таблицями, ключами та зв'язками між ними [8], [9], [10]. Для документування REST API може використовуватися OpenAPI та SpringDoc OpenAPI, які дозволяють описувати endpoint'и і перевіряти їх через Swagger UI [11], [12]. Для контролю стану застосунку може застосовуватися Spring Boot Actuator [13].

1.3 Опис основних користувачів системи

У системі управління онлайн-курсами можна виділити кілька основних груп користувачів, кожна з яких має власні задачі.

Першим користувачем є адміністратор системи. Він відповідає за загальне керування даними: додає викладачів, студентів, курси та навчальні модулі, а також редагує або видаляє записи. Для адміністратора важливо мати доступ до повної інформації про всі сутності системи.

Другим користувачем є викладач. Він пов'язаний з курсами, які веде, і може бути відповідальним за навчальний курс. Для нього важливо, щоб у системі зберігалася інформація про назву курсу, опис, рівень складності, дату створення та навчальні модулі.

Третім користувачем є студент. Він записується на курси та проходить навчання. Для студента важливими є доступ до інформації про курси, можливість запису на курс і збереження даних про прогрес навчання. Інформація про запис студента зберігається через сутність Enrollment.

Також окремо можна виділити API-клієнта. Це може бути frontend-застосунок, мобільний застосунок або інструмент тестування, наприклад Postman чи Swagger UI. API-клієнт надсилає HTTP-запити до backend-сервісу, а сервіс

повертає відповідь у вигляді даних. Такий підхід дозволяє відокремити серверну логіку від клієнтського інтерфейсу.

1.4 Постановка задачі курсової роботи

Метою курсової роботи є розробка backend-сервісу для управління онлайн-курсами з використанням Java, Spring Boot, Spring Data JPA, Hibernate ORM та PostgreSQL. Розроблений сервіс повинен забезпечувати збереження, обробку та отримання інформації про викладачів, курси, студентів, навчальні модулі та записи студентів на курси.

Об'єктом дослідження є процес розробки серверної частини інформаційної системи для управління онлайн-курсами. Предметом дослідження є моделі, зв'язки, REST API та програмні засоби, які використовуються для реалізації backend-сервісу.

Для досягнення поставленої мети необхідно виконати такі задачі: проаналізувати предметну область онлайн-курсів; визначити основні сутності системи; описати користувачів і функціональні вимоги; спроектувати зв'язки між сутностями; створити структуру бази даних; реалізувати REST API; забезпечити валідацію даних; реалізувати обробку помилок; додати пошук і аналітичні endpoint'и; перевірити працездатність системи за допомогою тестування API.

Загальна модель предметної області включає п'ять основних сутностей: Instructor, Course, Student, Enrollment та Module. Сутність Instructor використовується для збереження інформації про викладачів. Course описує онлайн-курс. Student зберігає інформацію про студентів. Enrollment фіксує запис студента на курс. Module описує навчальні модулі, які входять до складу курсу.

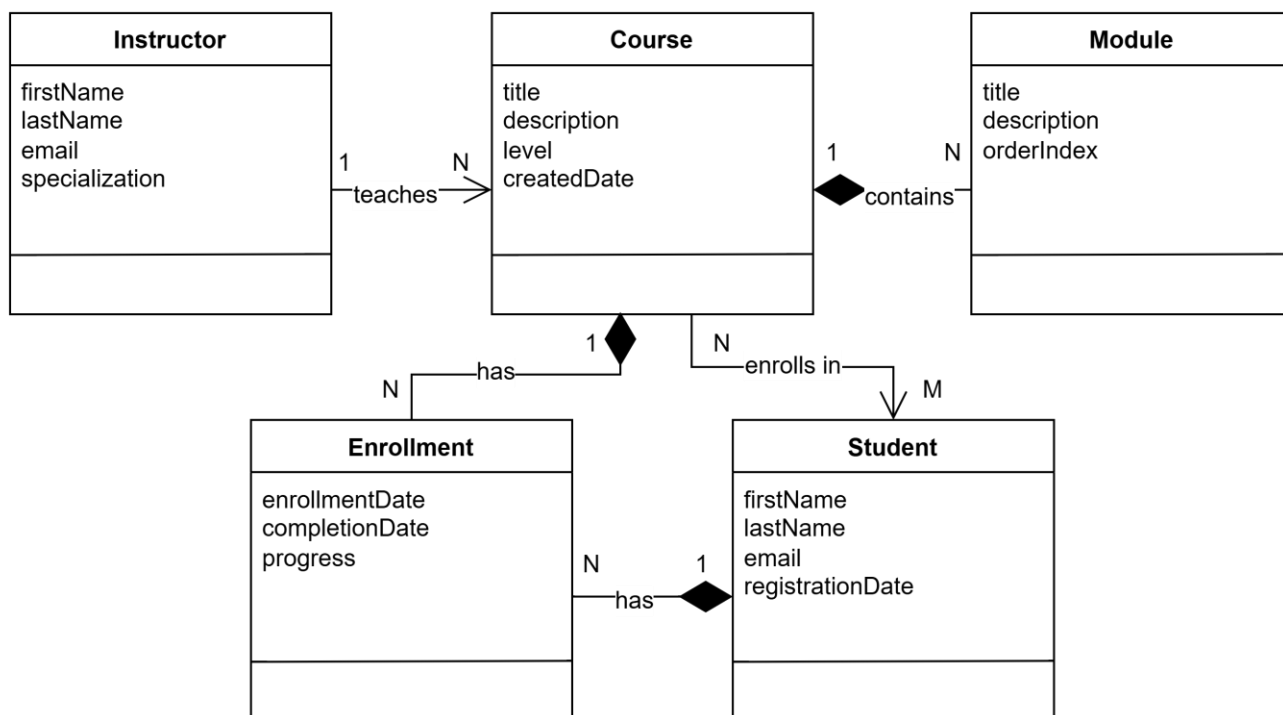


Рисунок 1.1 – Узагальнена діаграма сутностей backend-сервісу управління онлайн-курсами

Як показано на рисунку 1.1, викладач пов’язаний з курсом зв’язком «teaches», тобто викладач веде курс. Курс пов’язаний з навчальним модулем зв’язком «contains», тобто курс містить навчальні модулі. Студент пов’язується з курсом через сутність Enrollment, яка зберігає інформацію про запис студента на курс, дату запису, дату завершення та прогрес навчання.

Висновки до розділу 1

У першому розділі було розглянуто предметну область онлайн-курсів та визначено основні процеси, які має підтримувати backend-сервіс. Було встановлено, що система повинна працювати з викладачами, курсами, студентами, навчальними модулями та записами студентів на курси.

Також було проаналізовано функціональні та нефункціональні вимоги до системи. Визначено, що backend-сервіс повинен підтримувати CRUD-операції,

зв'язки між сутностями, пошук, аналітичні endpoint'и, валідацію даних, обробку помилок і перевірку стану застосунку.

У результаті аналізу було сформульовано мету курсової роботи, об'єкт, предмет та основні задачі розробки. Отримані результати є основою для подальшого проєктування архітектури backend-сервісу, структури бази даних, REST API та реалізації програмного продукту.

2 ПРОЄКТУВАННЯ BACKEND–СЕРВІСУ УПРАВЛІННЯ ОНЛАЙН–КУРСАМИ

2.1 Загальна архітектура програмного забезпечення

Backend–сервіс управління онлайн–курсами проєктується як серверна частина системи, яка приймає HTTP–запити, обробляє дані, виконує бізнес–логіку та взаємодіє з базою даних. Клієнтом може бути вебзастосунок, мобільний застосунок або інструмент для тестування API. Такий підхід дозволяє відокремити інтерфейс користувача від внутрішньої логіки системи та забезпечити більш зручну підтримку програмного продукту.

Для розробки використовується мова програмування Java, яка дозволяє будувати програму на основі об’єктно–орієнтованого підходу [1]. Основою застосунку є Spring Boot, що спрощує створення та запуск backend–сервісів, а також підтримує розробку REST API [2]–[4]. Для роботи з базою даних використовується Spring Data JPA, а для об’єктно–реляційного відображення – Hibernate ORM [6], [7]. Завдяки цим технологіям можна зручно реалізувати збереження, пошук, оновлення та видалення даних.

Архітектура системи має багаторівневу структуру. У ній виділено контролерний шар, сервісний шар, репозиторний шар, шар сутностей, шар DTO–класів та шар обробки помилок. Такий поділ дозволяє зробити структуру проєкту зрозумілою та логічною.

Контролери приймають HTTP–запити та повертають відповіді клієнту. Сервіси містять бізнес–логіку системи: створення, оновлення, видалення, пошук даних і перевірку бізнес–правил. Репозиторії відповідають за доступ до бази даних. Сутності описують основні об’єкти предметної області, а DTO–класи використовуються для передачі даних між клієнтом і сервером. Окремий шар обробки помилок забезпечує повернення зрозумілих повідомлень у разі

некоректних запитів або відсутності потрібних даних.

Загальна логіка роботи системи така: клієнт надсилає запит до REST–контролера, контролер передає дані до сервісу, сервіс виконує необхідну логіку та звертається до репозиторію, після чого результат повертається клієнту.

2.2 Проєктування сутностей системи

У backend–сервісі проєктуються п’ять основних сутностей: Instructor, Course, Student, Enrollment та CourseModule.

Сутність Instructor зберігає інформацію про викладача: ім’я, прізвище, email та спеціалізацію. Сутність Course описує онлайн–курс: назву, опис, рівень складності та дату створення. Сутність Student містить дані про студента: ім’я, прізвище, email та дату реєстрації.

Сутність Enrollment використовується для запису студента на курс. Вона зберігає дату запису, дату завершення та прогрес навчання. Сутність CourseModule описує навчальний модуль курсу, його назву, опис і порядковий номер.

Таблиця 2.1 – Основні сутності системи

Сутність	Призначення
Instructor	Зберігає інформацію про викладачів
Course	Описує онлайн–курси
Student	Зберігає інформацію про студентів
Enrollment	Фіксує запис студента на курс
CourseModule	Описує навчальні модулі курсів

Таблиця 2.2 – Поля сутностей системи

Сутність	Основні поля
Instructor	id, firstName, lastName, email, specialization
Course	id, title, description, level, createdAt
Student	id, firstName, lastName, email, registrationDate
Enrollment	id, enrollmentDate, completionDate, progress
CourseModule	id, title, description, orderIndex

2.3 Проектування зв'язків між сутностями

Між сутностями системи передбачено зв'язки, які відображають логіку предметної області онлайн–курсів.

Один викладач може вести багато курсів, тому між Instructor і Course встановлено зв'язок один до багатьох. Один курс може містити багато навчальних модулів, тому між Course і CourseModule також встановлено зв'язок один до багатьох.

Між Student і Course існує зв'язок багато до багатьох, оскільки один студент може бути записаний на декілька курсів, а один курс може мати багато студентів. Для реалізації цього зв'язку використовується проміжна сутність Enrollment. Вона дозволяє не тільки пов'язати студента з курсом, а й зберегти додаткову інформацію про навчання.

Таблиця 2.3 – Зв'язки між сутностями

Зв'язок	Кратність	Опис
Instructor – Course	1 : N	Один викладач веде багато курсів
Course – CourseModule	1 : N	Один курс містить багато модулів
Student – Enrollment	1 : N	Один студент має багато записів
Course – Enrollment	1 : N	Один курс має багато записів
Student – Course	M : N	Реалізується через Enrollment

2.4 Проектування структури бази даних

Для збереження даних використовується реляційна база даних PostgreSQL. Вона дозволяє зберігати інформацію у вигляді таблиць і підтримує зв'язки між ними за допомогою ключів [8]–[10].

Структура бази даних формується відповідно до основних сутностей системи. Для кожної сутності створюється окрема таблиця. Таблиця instructor зберігає викладачів, course – курси, student – студентів, course_module – навчальні

модулі, enrollment – записи студентів на курси.

Структуру бази даних із первинними та зовнішніми ключами наведено на рисунку 2.1.

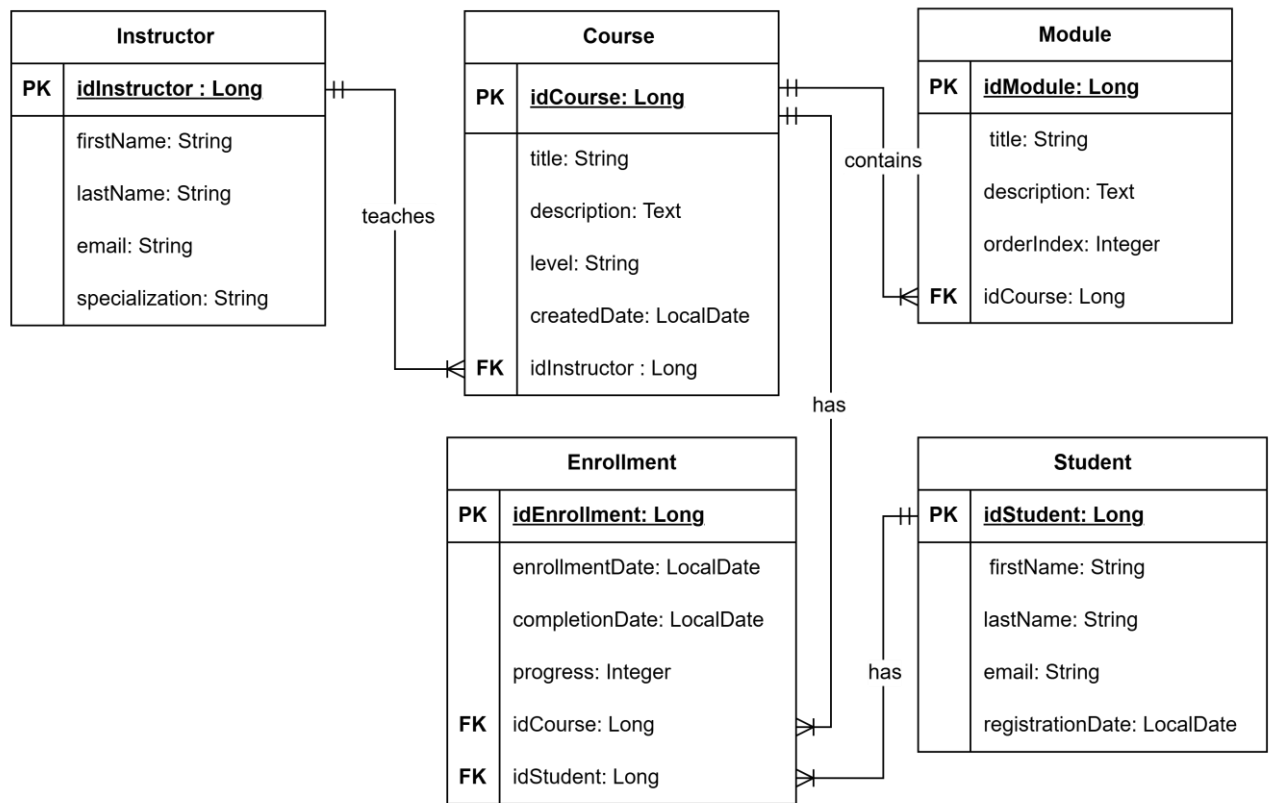


Рисунок 2.1 – Структура бази даних backend-сервісу управління онлайн-курсами

Як показано на рисунку 2.1, таблиця course пов’язана з таблицею instructor, таблиця course_module пов’язана з таблицею course, а таблиця enrollment пов’язана одночасно з таблицями student і course.

Таблиця 2.4 – Таблиці бази даних

Таблиця	Призначення
instructor	Дані викладачів
course	Дані онлайн-курсів
student	Дані студентів
course_module	Дані навчальних модулів
enrollment,	Дані записів студентів на курси

2.5 Проектування REST API

REST API є основним способом взаємодії клієнта з backend-сервісом. У системі передбачено endpoint'и для роботи з викладачами, курсами, студентами, записами на курси та навчальними модулями.

Таблиця 2.5 – REST API для викладачів

Метод	Endpoint	Призначення
POST	/instructors	Створити викладача
GET	/instructors	Отримати всіх викладачів
GET	/instructors/{id}	Отримати викладача за id
PUT	/instructors/{id}	Оновити викладача
DELETE	/instructors/{id}	Видалити викладача
GET	/instructors/{id}/courses	Отримати курси викладача

Таблиця 2.6 – REST API для курсів

Метод	Endpoint	Призначення
POST	/courses	Створити курс
GET	/courses	Отримати всі курси
GET	/courses/{id}	Отримати курс за id
PUT	/courses/{id}	Оновити курс
DELETE	/courses/{id}	Видалити курс
GET	/courses/{id}/modules	Отримати модулі курсу
GET	/courses/{id}/students	Отримати студентів курсу

Таблиця 2.7 – REST API для студентів

Метод	Endpoint	Призначення
POST	/students	Створити студента
GET	/students	Отримати всіх студентів
GET	/students/{id}	Отримати студента за id
PUT	/students/{id}	Оновити студента
DELETE	/students/{id}	Видалити студента
GET	/students/{id}/courses	Отримати курси студента

Таблиця 2.8 – REST API для записів на курси та модулів

Метод	Endpoint	Призначення
POST	/enrollments	Записати студента на курс
GET	/enrollments	Отримати всі записи
GET	/enrollments/{id}	Отримати запис за id
PUT	/enrollments/{id}/progress	Оновити прогрес
DELETE	/enrollments/{id}	Видалити запис
POST	/modules	Створити модуль
GET	/modules	Отримати всі модулі
GET	/modules/{id}	Отримати модуль за id
PUT	/modules/{id}	Оновити модуль
DELETE	/modules/{id}	Видалити модуль
POST	/courses/{id}/modules/{moduleId}	Додати модуль до курсу
DELETE	/courses/{id}/modules/{moduleId}	Видалити модуль з курсу

Для документування API передбачено використання OpenAPI та SpringDoc OpenAPI, які дозволяють описувати endpoint'и, параметри запитів і відповіді сервера [11], [12].

2.6 Проєктування повнотекстового пошуку

У системі передбачено пошук курсів і студентів. Пошук курсів виконується за назвою та описом. Також передбачено розширений пошук за ключовим словом і рівнем складності курсу.

Пошук студентів виконується за іменем, прізвищем або електронною поштою. Такий підхід дозволяє швидко знайти потрібного студента без перегляду всього списку записів.

Таблиця 2.9 – REST API для пошуку

Метод	Endpoint	Призначення
GET	/search/courses?query=value	Пошук курсів
GET	/search/courses/advanced?query=value&level=value	Розширений пошук курсів
GET	/search/students?query=value	Пошук студентів

Пошук проєктується без урахування регістру символів. Результатом запиту є список знайдених курсів або студентів у форматі DTO–відповіді.

2.7 Проєктування аналітичних запитів

Окрім CRUD–операцій і пошуку, у backend–сервісі передбачено аналітичні endpoint’и. Вони потрібні для отримання узагальненої інформації про роботу системи.

Таблиця 2.10 – REST API для аналітичних запитів

Метод	Endpoint	Призначення
GET	/analytics/courses/count	Отримати кількість курсів
GET	/analytics/students/count	Отримати кількість студентів
GET	/analytics/enrollments/active	Отримати кількість активних записів
GET	/analytics/enrollments/completed	Отримати кількість завершених записів
GET	/analytics/courses/by-level	Отримати кількість курсів за рівнями

Також у системі передбачено endpoint’и для перевірки стану сервісу та перегляду метрик.

Таблиця 2.11 – Endpoint’и стану сервісу

Метод	Endpoint	Призначення
GET	/actuator/health	Перевірити стан сервісу
GET	/actuator/metrics	Отримати список метрик
GET	/actuator/prometheus	Отримати метрики у текстовому форматі

Висновки до розділу 2

У другому розділі було виконано проєктування backend–сервісу управління онлайн–курсами. Було визначено архітектуру системи, основні шари застосунку, сутності, їхні поля та зв’язки.

Також було спроектовано структуру бази даних, REST API, пошук курсів і

студентів, аналітичні endpoint'и та endpoint'и перевірки стану сервісу. Запропоновані рішення є основою для реалізації програмного продукту, яка розглядається у третьому розділі.

3 РЕАЛІЗАЦІЯ ПРОГРАМНОГО ПРОДУКТУ

3.1 Використані технології та інструменти

Для реалізації backend-сервісу управління онлайн-курсами було використано мову програмування Java, фреймворк Spring Boot, Spring Data JPA, Hibernate ORM та базу даних PostgreSQL. Java застосовується для створення основних класів системи, Spring Boot – для запуску backend-застосунку та створення REST API, Spring Data JPA – для роботи з репозиторіями, а Hibernate ORM – для зв'язку Java-класів із таблицями бази даних [1]–[7].

Для документування API використано OpenAPI та SpringDoc OpenAPI. Це дозволяє описувати endpoint'и, параметри запитів і відповіді сервера [11], [12]. Також у системі реалізовано endpoint'и для перевірки стану сервісу та отримання метрик.

3.2 Реалізація сутностей instructor, course, student, enrollment та module

У програмному продукті реалізовано п'ять основних сутностей: Instructor, Course, Student, Enrollment та CourseModule.

Сутність Course описує онлайн-курс і містить назву, опис, рівень складності, дату створення та зв'язки з викладачем, навчальними модулями і записами студентів на курс.

Лістинг 3.1 – Фрагмент реалізації сутності Course

```
@Entity
public class Course {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    @Column(columnDefinition = "TEXT")
    private String description;
    private String level;
    private LocalDate createdAt;
```

```

@ManyToOne
@JoinColumn(name = "instructor_id")
private Instructor instructor;

@OneToMany(mappedBy = "course")
private List<CourseModule> modules;

@OneToMany(mappedBy = "course")
private List<Enrollment> enrollments;
}

```

Сутність Enrollment використовується для запису студента на курс. Вона зберігає дату запису, дату завершення, прогрес навчання та зв'язки зі студентом і курсом.

Лістинг 3.2 – Фрагмент реалізації сутності Enrollment

```

@Entity
public class Enrollment {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private LocalDate enrollmentDate;
    private LocalDate completionDate;
    private Integer progress;

    @ManyToOne
    @JoinColumn(name = "student_id")
    private Student student;

    @ManyToOne
    @JoinColumn(name = "course_id")
    private Course course;
}

```

Таким чином, сутності формують основу структури даних системи та використовуються для створення відповідних таблиць у базі даних.

3.3 Реалізація репозиторіїв

Для доступу до бази даних у системі створено репозиторії. Вони наслідують `JpaRepository`, що дає змогу виконувати стандартні операції збереження, пошуку, оновлення та видалення даних.

У `CourseRepository` реалізовано пошук курсів за назвою або описом, розширений пошук за ключовим словом і рівнем складності, а також аналітичний запит для підрахунку курсів за рівнями.

Лістинг 3.3 – Фрагмент реалізації `CourseRepository`

```
@Repository
public interface CourseRepository extends JpaRepository<Course, Long> {
    List<Course>
    findByTitleContainingIgnoreCaseOrDescriptionContainingIgnoreCase(String title,
    String description);

    @Query("""
        select c from Course c
        where (:level is null or lower(c.level) = lower(:level))
        and (lower(c.title) like lower(concat('%', :query, '%'))
            or lower(c.description) like lower(concat('%', :query, '%')))
        """)
    List<Course> searchByQueryAndLevel(@Param("query") String query,
    @Param("level") String level);

    @Query("select c.level, count(c) from Course c group by c.level")
    List<Object[]> countCoursesByLevel();
}
```

3.4 Реалізація сервісного шару

Сервісний шар містить бізнес-логіку системи: створення, оновлення, видалення даних та перевірку основних бізнес-правил.

Наприклад, у `EnrollmentService` реалізовано створення запису студента на курс. Перед збереженням перевіряється дата завершення курсу, наявність студента і курсу, а також те, чи студент уже не записаний на цей курс.

Лістинг 3.4 – Фрагмент реалізації створення запису на курс

```
public Enrollment create(EnrollmentCreateRequest request) {
    validateCompletionDate(request.enrollmentDate(), request.completionDate());
    if (repository.existsByStudentIdAndCourseId(request.studentId(),
    request.courseId())) {
        throw new CourseEnrollmentException("Student is already enrolled in this
    course");
    }

    Student student = studentRepository.findById(request.studentId())
        .orElseThrow(() -> new ResourceNotFoundException("Student",
    request.studentId()));
}
```

```

    Course course = courseRepository.findById(request.courseId())
        .orElseThrow(() -> new ResourceNotFoundException("Course",
request.courseId()));

    Enrollment enrollment = new Enrollment();
    enrollment.setStudent(student);
    enrollment.setCourse(course);
    enrollment.setEnrollmentDate(request.enrollmentDate());
    enrollment.setCompletionDate(request.completionDate());
    enrollment.setProgress(request.progress());
    return repository.save(enrollment);
}

```

3.5 Реалізація rest-контролерів

REST-контролери відповідають за обробку HTTP-запитів. У системі створено контролери для курсів, викладачів, студентів, навчальних модулів, записів на курси, пошуку, аналітики та перевірки стану сервісу.

Фрагмент CourseController наведено у лістингу 3.5.

Лістинг 3.5 – Фрагмент реалізації CourseController

```

@RestController
@RequestMapping("/courses")
@Tag(name = "Курси", description = "Управління онлайн-курсами, їх викладачами, модулями та студентами")
public class CourseController {

    @Autowired
    private CourseService service;

    @PostMapping
    public ResponseEntity<CourseResponse> create(@Valid @RequestBody
CourseCreateRequest request) {
        return ResponseEntity.ok(toResponse(service.create(request)));
    }

    @GetMapping
    public ResponseEntity<List<CourseResponse>> getAll() {
        return
ResponseEntity.ok(service.getAll().stream().map(this::toResponse).toList());
    }

    @GetMapping("/{id}")
    public ResponseEntity<CourseResponse> getById(@PathVariable Long id) {
        return ResponseEntity.ok(toResponse(service.getById(id)));
    }
}

```

Цей контролер реалізує основні endpoint'и для створення курсів, отримання списку курсів і пошуку курсу за ідентифікатором.

3.6 Реалізація DTO та валідації даних

DTO-класи використовуються для передачі даних між клієнтом і сервером. Вони дозволяють не передавати напряду entity-класи та контролювати структуру запитів і відповідей.

У DTO-класах використовується валідація. Наприклад, у CourseCreateRequest перевіряється назва курсу, опис, рівень складності, дата створення та ідентифікатор викладача.

Лістинг 3.6 – Фрагмент реалізації CourseCreateRequest

```
@Schema(description = "Дані для створення нового онлайн-курсу")
public record CourseCreateRequest(
    @NotBlank(message = "Course title is required")
    @Size(min = 3, max = 150, message = "Course title must contain from 3 to
150 characters")
    String title,

    @NotBlank(message = "Course description is required")
    @Size(min = 20, max = 2000, message = "Course description must contain
from 20 to 2000 characters")
    String description,

    @NotBlank(message = "Course level is required")
    @Pattern(regexp = "BEGINNER|INTERMEDIATE|ADVANCED", message = "Course
level must be BEGINNER, INTERMEDIATE or ADVANCED")
    String level,

    @NotNull(message = "Created date is required")
    @PastOrPresent(message = "Created date cannot be in the future")
    LocalDate createdAt,

    @NotNull(message = "Instructor id is required")
    @Positive(message = "Instructor id must be positive")
    Long instructorId
) {
}
```

Завдяки валідації система може відхиляти некоректні запити ще до виконання бізнес-логіки.

3.7 Реалізація зв'язків між курсами, студентами, викладачами та модулями

Зв'язки між сутностями реалізовано за допомогою JPA-анотацій. Курс має зв'язок із викладачем, навчальними модулями та записами студентів. Запис Enrollment пов'язує студента з конкретним курсом.

У системі студент не записується на курс напряму. Для цього використовується проміжна сутність Enrollment, яка дозволяє зберігати прогрес навчання, дату запису та дату завершення курсу.

Також реалізовано зв'язок між курсом і навчальними модулями. Кожен модуль має посилання на курс, до якого він належить.

3.8 Реалізація пошуку курсів і студентів

Пошук реалізовано в TextSearchController. Для курсів передбачено пошук за назвою та описом, а також розширений пошук за ключовим словом і рівнем складності. Для студентів реалізовано пошук за іменем, прізвищем або email.

Лістинг 3.7 – Фрагмент реалізації TextSearchController

```
@GetMapping("/courses")
public ResponseEntity<List<CourseResponse>> searchCourses(@RequestParam String
query) {
    return ResponseEntity.ok(courseRepository

        .findByTitleContainingIgnoreCaseOrDescriptionContainingIgnoreCase(query, query)
            .stream()
            .map(this::toCourseResponse)
            .toList());
}

@GetMapping("/students")
public ResponseEntity<List<StudentResponse>> searchStudents(@RequestParam String
query) {
    return ResponseEntity.ok(studentRepository

        .findByFirstNameContainingIgnoreCaseOrLastNameContainingIgnoreCaseOrEmailContainin
gIgnoreCase(
            query,
            query,
            query
        )
    )
}
```

```

        .stream()
        .map(this::toStudentResponse)
        .toList());
}

```

Пошук виконується без урахування регістру символів, тому користувач може вводити запит у різному форматі.

3.9 Реалізація аналітичних endpoint'ів

Для отримання узагальненої інформації у системі реалізовано `AnalyticsController`. Він дозволяє отримувати кількість курсів, кількість студентів, кількість активних і завершених записів, а також статистику курсів за рівнем складності.

Лістинг 3.8 – Фрагмент реалізації `AnalyticsController`

```

@GetMapping("/courses/count")
public ResponseEntity<Long> coursesCount() {
    return ResponseEntity.ok(courseRepository.count());
}

@GetMapping("/students/count")
public ResponseEntity<Long> studentsCount() {
    return ResponseEntity.ok(studentRepository.count());
}

@GetMapping("/enrollments/active")
public ResponseEntity<Long> active() {
    return ResponseEntity.ok(enrollmentRepository.countByCompletionDateIsNull());
}

@GetMapping("/enrollments/completed")
public ResponseEntity<Long> completed() {
    return
    ResponseEntity.ok(enrollmentRepository.countByCompletionDateIsNotNull());
}

```

3.10 Налаштування actuator та метрик сервісу

Для перевірки стану backend-сервісу реалізовано `ActuatorController`. Він містить endpoint'и `/actuator/health`, `/actuator/metrics` та `/actuator/prometheus`.

Лістинг 3.9 – Фрагмент реалізації ActuatorController

```

@GetMapping("/health")
public ResponseEntity<Map<String, Object>> health() {
    return ResponseEntity.ok(Map.of(
        "status", "UP",
        "service", "labwork-2",
        "timestamp", LocalDateTime.now()
    ));
}
@GetMapping("/metrics")
public ResponseEntity<Map<String, Object>> metrics() {
    return ResponseEntity.ok(Map.of(
        "availableMetrics", List.of(
            "courses.count",
            "students.count",
            "enrollments.active",
            "enrollments.completed",
            "courses.byLevel"
        )
    ));
}

```

Endpoint `/actuator/health` показує, що сервіс працює, а `/actuator/metrics` повертає список доступних метрик.

3.11 Обробка помилок у системі

У системі реалізовано централізовану обробку помилок за допомогою `GlobalExceptionHandler`. Він перехоплює помилки валідації, некоректні запити, відсутність ресурсу, конфлікти бізнес–правил та внутрішні помилки сервера.

Фрагмент обробки помилок наведено у лістингу 3.10.

Лістинг 3.10 – Фрагмент реалізації GlobalExceptionHandler

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiErrorResponse> handleValidation(
        MethodArgumentNotValidException ex,
        HttpServletRequest request) {
        Map<String, String> errors = new LinkedHashMap<>();
        for (FieldError fieldError : ex.getBindingResult().getFieldErrors()) {
            errors.put(fieldError.getField(), fieldError.getDefaultMessage());
        }
        return build(HttpStatus.BAD_REQUEST, "Validation failed",
            request.getRequestURI(), errors);
    }
}

```

```

    @ExceptionHandler({ResourceNotFoundException.class,
NoSuchElementException.class})
    public ResponseEntity<ApiErrorResponse> handleNotFound(Exception ex,
HttpServletRequest request) {
        return build(HttpStatus.NOT_FOUND, ex.getMessage(),
request.getRequestURI(), null);
    }
}

```

Завдяки цьому клієнт отримує зрозумілу відповідь у разі помилки, а система має єдиний формат обробки винятків.

Висновки до розділу 3

У третьому розділі було описано реалізацію backend-сервісу управління онлайн-курсами. Було розглянуто використані технології, реалізацію сутностей, репозиторіїв, сервісів, REST-контролерів, DTO-класів, пошуку, аналітичних endpoint'ів, метрик та обробки помилок.

У результаті реалізовано backend-сервіс, який дозволяє працювати з викладачами, курсами, студентами, навчальними модулями та записами студентів на курси через REST API.

4 ТЕСТУВАННЯ ТА НАЛАГОДЖЕННЯ BACKEND-SERVICE

4.1 Тестування API викладачів

Тестування backend-сервісу було виконано шляхом надсилання HTTP-запитів до REST API. Основною метою тестування була перевірка правильності створення, отримання, оновлення та видалення даних.

Спочатку було перевірено роботу API викладачів. Для цього використовувалися endpoint'и POST /instructors, GET /instructors, GET /instructors/{id}, PUT /instructors/{id}, DELETE /instructors/{id} та GET /instructors/{id}/courses. Результати тестування API викладачів наведено на рисунках 4.1–4.6.

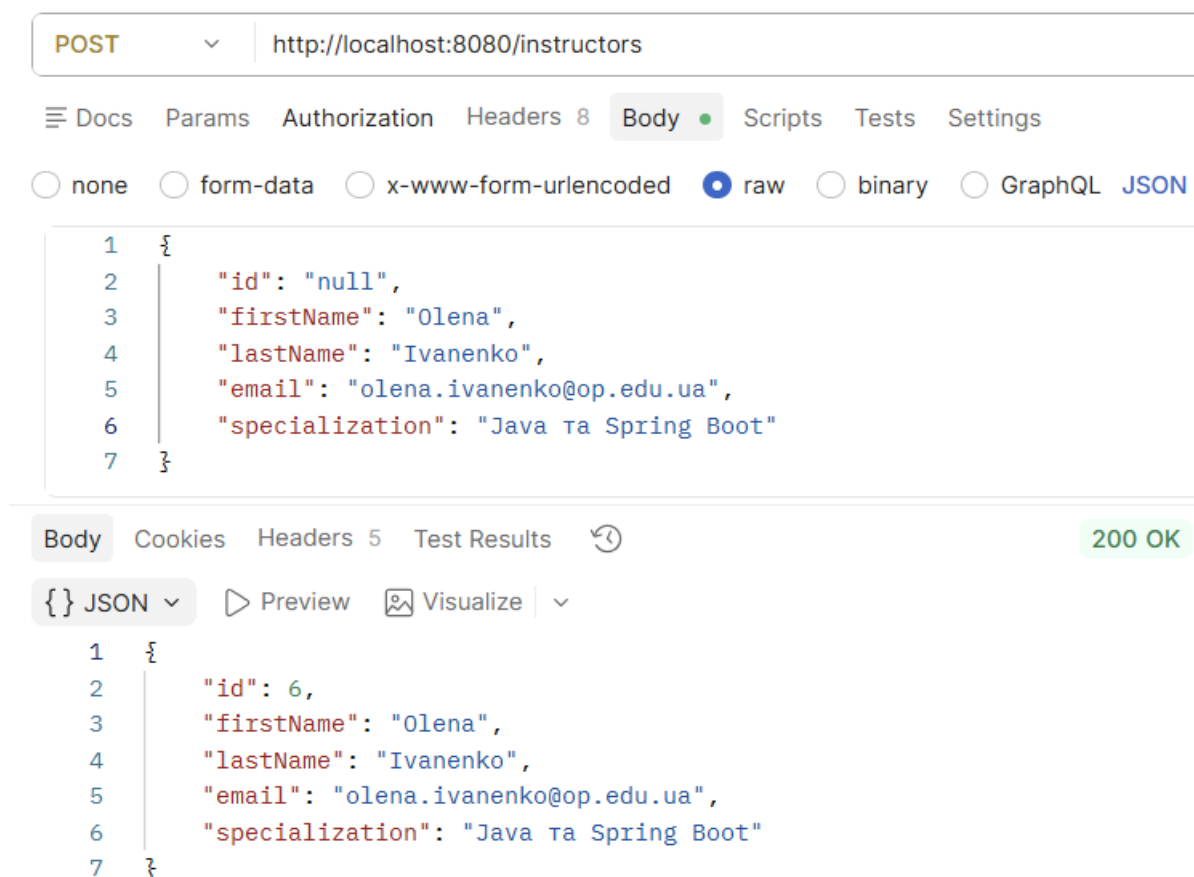


Рисунок 4.1 – Результат тестування створення викладача



Рисунок 4.2 – Результат тестування отримання списку викладачів

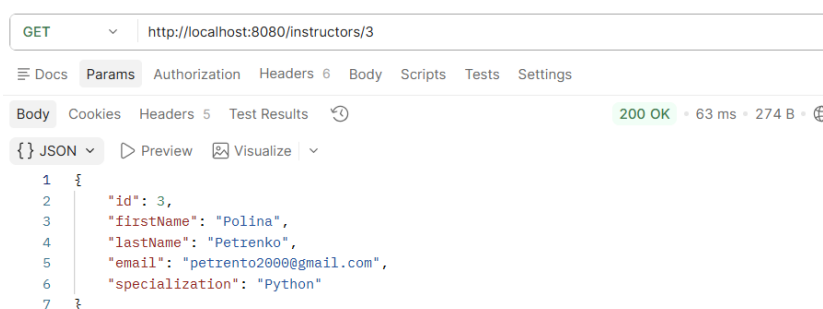


Рисунок 4.3 – Результат тестування отримання викладача за id

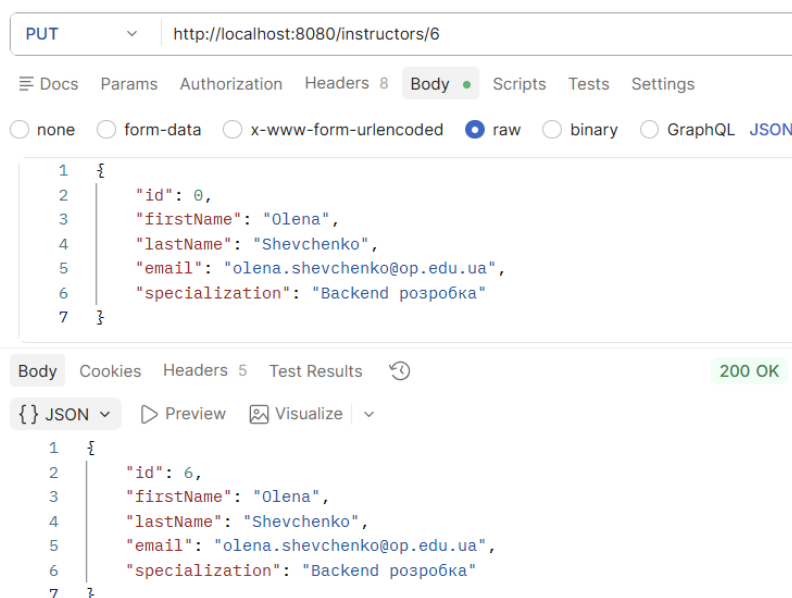


Рисунок 4.4 – Результат тестування оновлення даних викладача

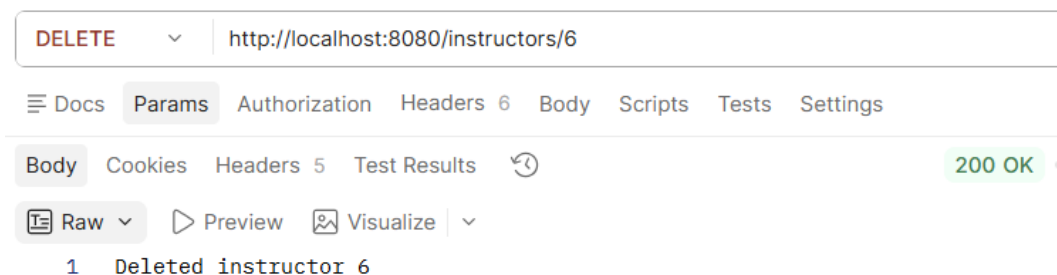


Рисунок 4.5 – Результат тестування видалення викладача

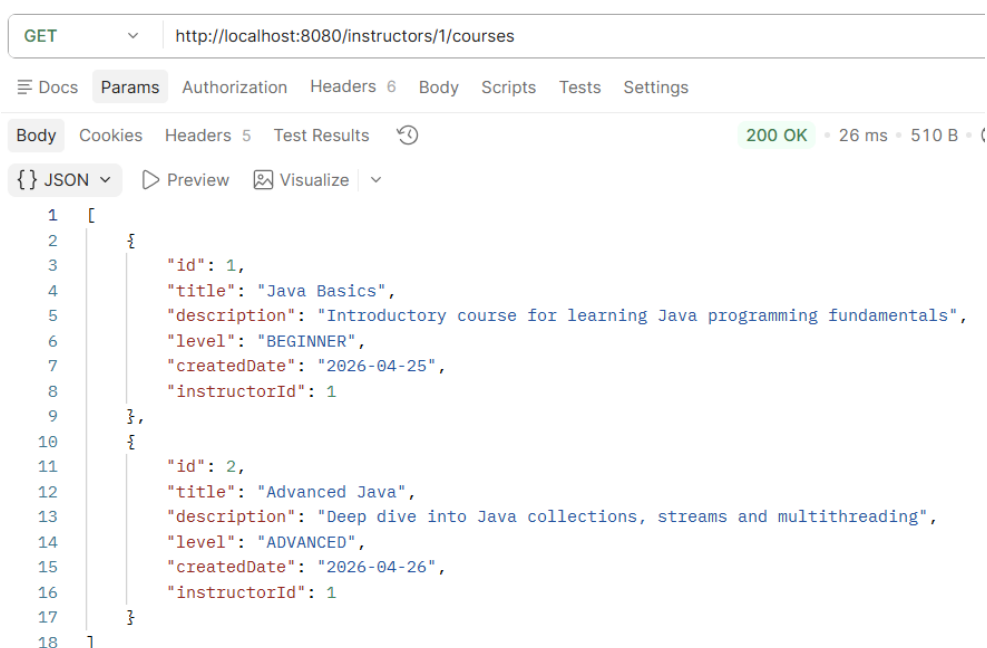


Рисунок 4.6 – Результат тестування отримання курсів викладача

У результаті тестування було встановлено, що API викладачів коректно обробляє основні CRUD-операції та дозволяє отримувати курси конкретного викладача.

4.2 Тестування API курсів

Під час тестування API курсів перевірялися основні операції з онлайн-курсами. Для цього використовувалися endpoint'и POST /courses, GET /courses, GET /courses/{id}, PUT /courses/{id}, DELETE /courses/{id}, GET

/courses/{id}/modules та GET /courses/{id}/students. Результати тестування API курсів наведено на рисунках 4.7–4.13.

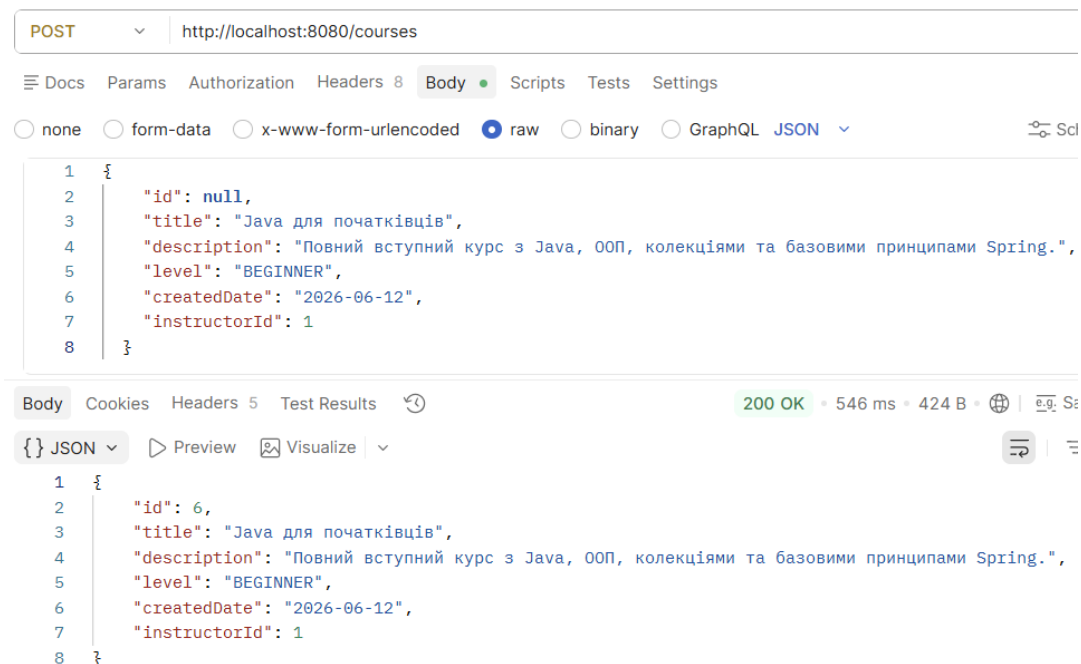


Рисунок 4.7 – Результат тестування створення курсу

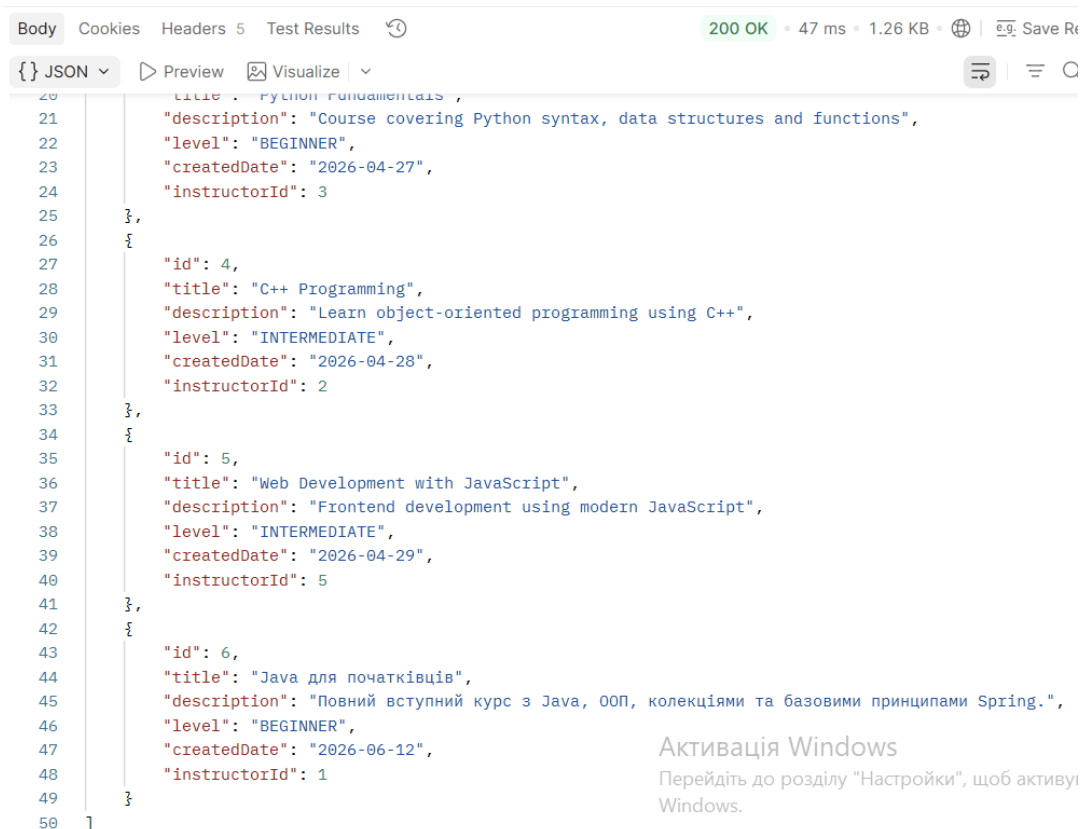


Рисунок 4.8 – Результат тестування отримання списку курсів

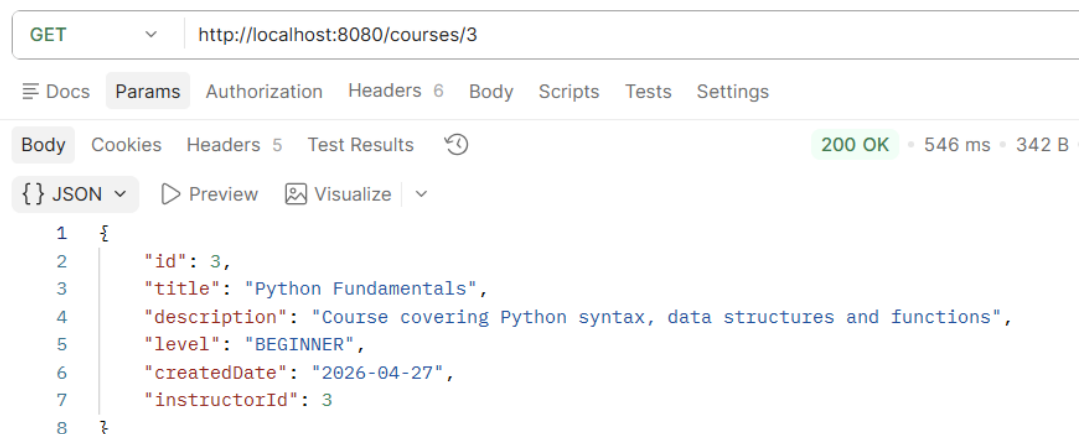


Рисунок 4.9 – Результат тестування отримання курсу за id

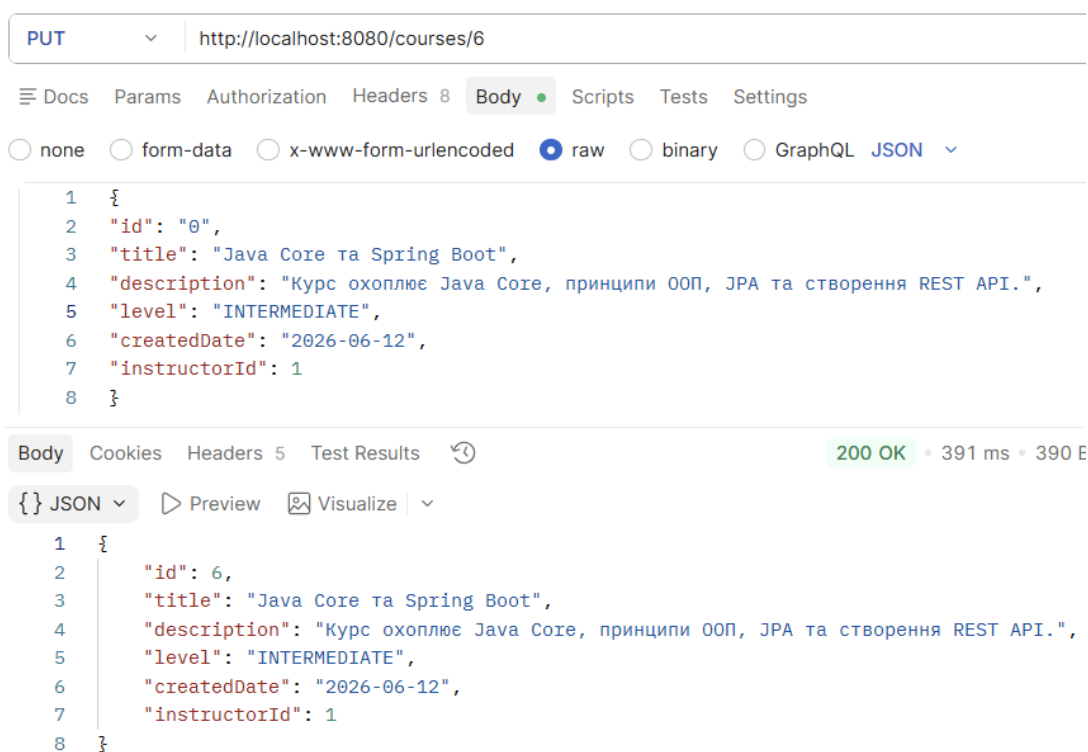


Рисунок 4.10 – Результат тестування оновлення курсу

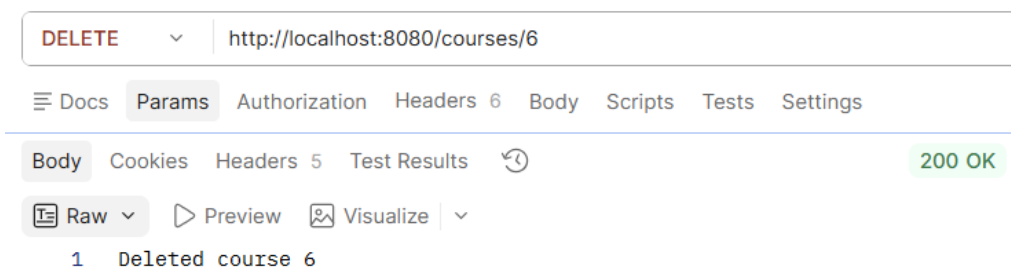


Рисунок 4.11 – Результат тестування видалення курсу

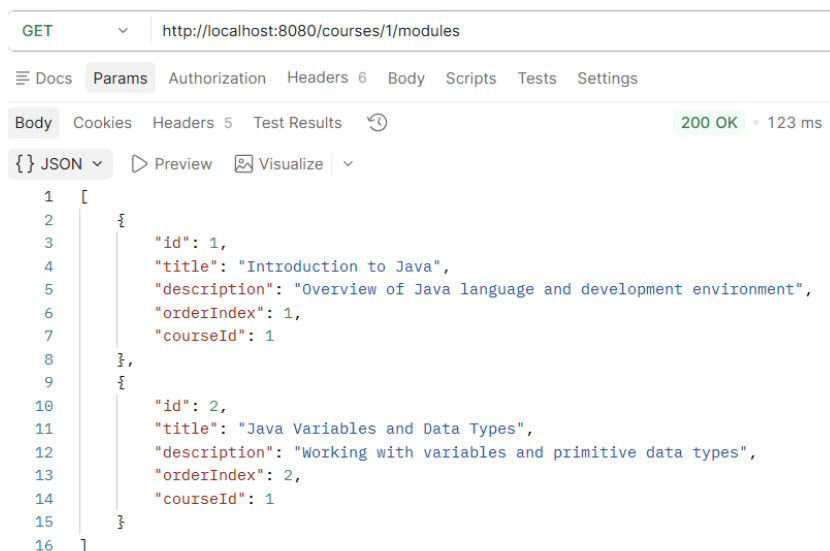


Рисунок 4.12 – Результат тестування отримання модулів курсу

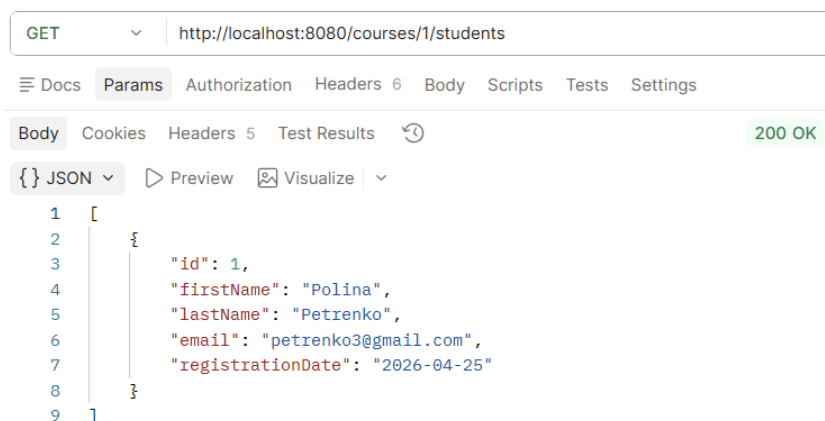


Рисунок 4.13 – Результат тестування отримання студентів курсу

У результаті перевірки було підтверджено, що API курсів дозволяє створювати, переглядати, оновлювати та видаляти курси, а також отримувати пов'язані з курсом модулі та записи студентів.

4.3 Тестування API студентів

API студентів тестувалося через endpoint'и POST /students, GET /students, GET /students/{id}, PUT /students/{id}, DELETE /students/{id} та GET /students/{id}/courses. Результати тестування наведено на рисунках 4.14–4.19.

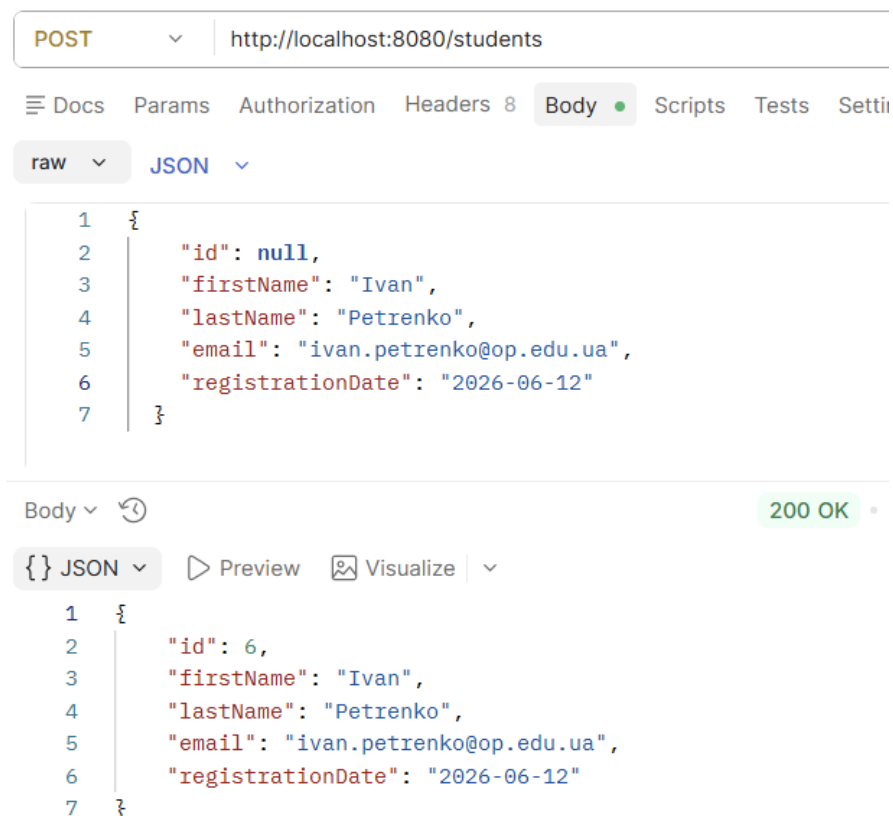


Рисунок 4.14 – Результат тестування створення студента



Рисунок 4.15 – Результат тестування отримання списку студентів

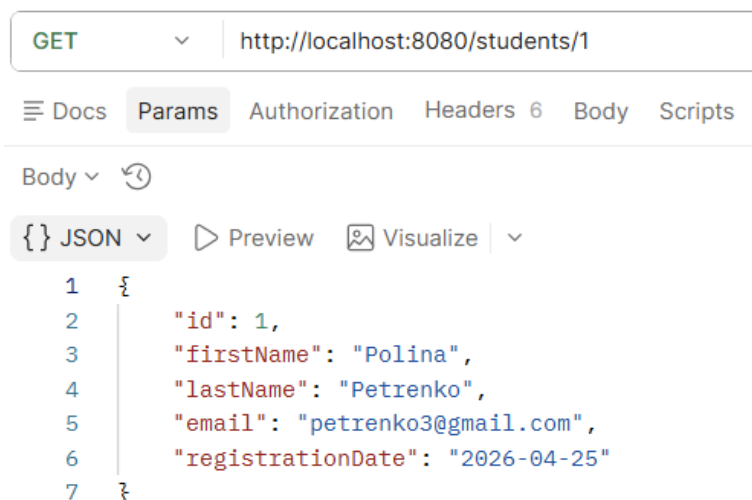


Рисунок 4.16 – Результат тестування отримання студента за id

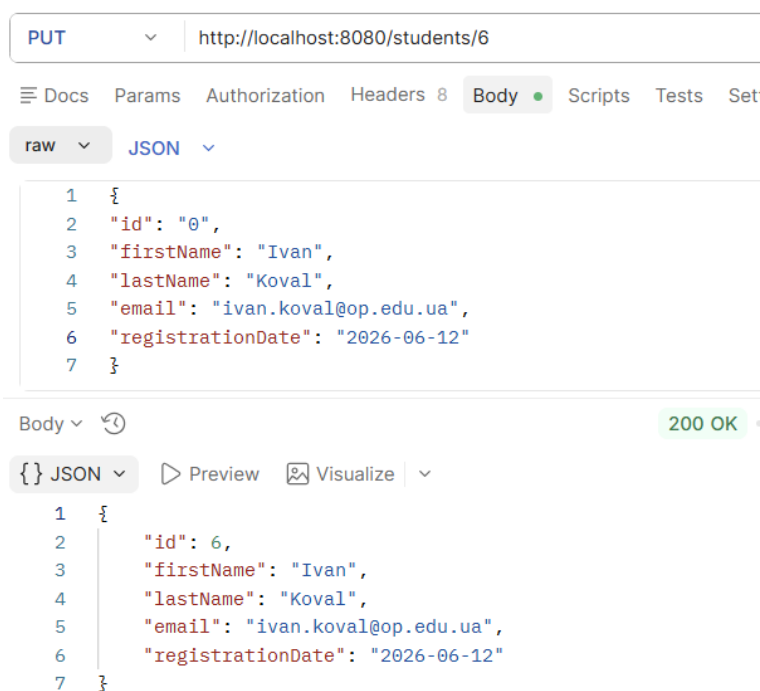


Рисунок 4.17 – Результат тестування оновлення даних студента

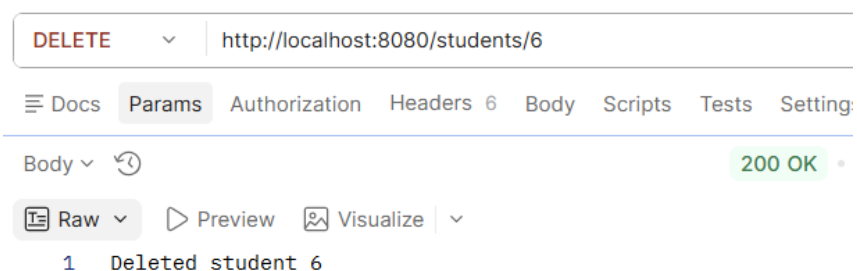


Рисунок 4.18 – Результат тестування видалення студента

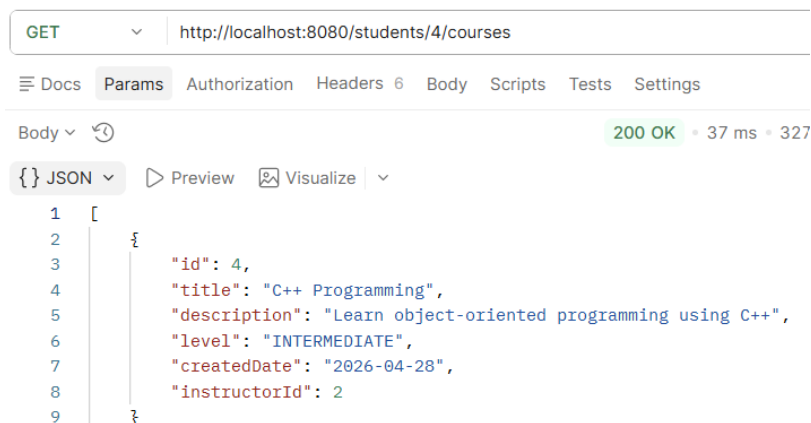


Рисунок 4.19 – Результат тестування отримання курсів студента

Після виконання запитів було встановлено, що дані студентів коректно створюються, зберігаються, оновлюються та видаляються.

4.4 Тестування запису студентів на курси

Для тестування запису студентів на курси використовувалися endpoint'и POST /enrollments, GET /enrollments, GET /enrollments/{id}, PUT /enrollments/{id}/progress та DELETE /enrollments/{id}. Результати тестування наведено на рисунках 4.20–4.24.

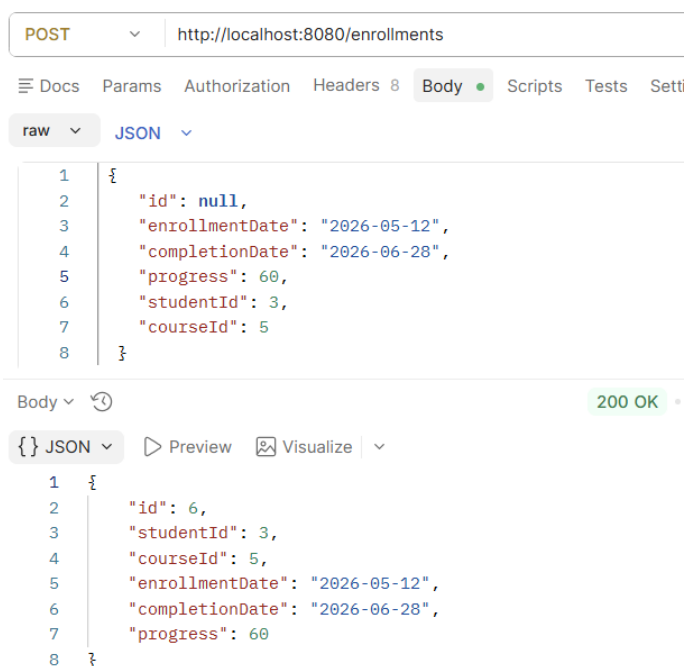


Рисунок 4.20 – Результат тестування запису студента на курс

```

27     "id": 4,
28     "studentId": 4,
29     "courseId": 4,
30     "enrollmentDate": "2026-04-10",
31     "completionDate": "2026-05-15",
32     "progress": 30
33   },
34   {
35     "id": 5,
36     "studentId": 5,
37     "courseId": 5,
38     "enrollmentDate": "2026-04-15",
39     "completionDate": "2026-05-20",
40     "progress": 65
41   },
42   {
43     "id": 6,
44     "studentId": 3,
45     "courseId": 5,
46     "enrollmentDate": "2026-05-12",
47     "completionDate": "2026-06-28",
48     "progress": 60
49   }
50 ]

```

Активация
Перейдіть до
Windows.

Рисунок 4.21 – Результат тестування отримання списку записів на курси

GET http://localhost:8080/enrollments/5

Docs Params Authorization Headers 6 Body Scripts Tests Settings

Body 200 OK

```

1 {
2   "id": 5,
3   "studentId": 5,
4   "courseId": 5,
5   "enrollmentDate": "2026-04-15",
6   "completionDate": "2026-05-20",
7   "progress": 65
8 }

```

Рисунок 4.22 – Результат тестування отримання запису на курс за id

PUT http://localhost:8080/enrollments/6/progress

Docs Params Authorization Headers 8 Body Scripts Tests Settings

raw JSON

```

1 {
2   "progress": 80
3 }

```

Body 200 OK

```

1 {
2   "id": 6,
3   "studentId": 3,
4   "courseId": 5,
5   "enrollmentDate": "2026-05-12",
6   "completionDate": "2026-06-28",
7   "progress": 80
8 }

```

Рисунок 4.23 – Результат тестування оновлення прогресу навчання

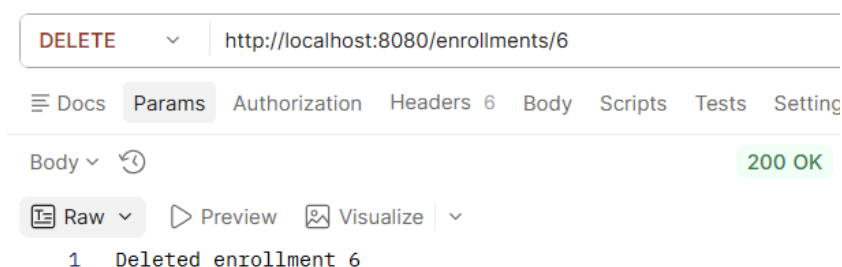


Рисунок 4.24 – Результат тестування видалення запису на курс

Під час тестування було перевірено, що студент може бути записаний на курс, прогрес навчання оновлюється коректно, а записи на курси можна переглядати та видаляти.

4.5 Тестування API навчальних модулів

Для перевірки навчальних модулів використовувалися endpoint'и POST /modules, GET /modules, GET /modules/{id}, PUT /modules/{id}, DELETE /modules/{id}, POST /courses/{id}/modules/{moduleId} та DELETE /courses/{id}/modules/{moduleId}. Результати тестування наведено на рисунках 4.25–4.31.

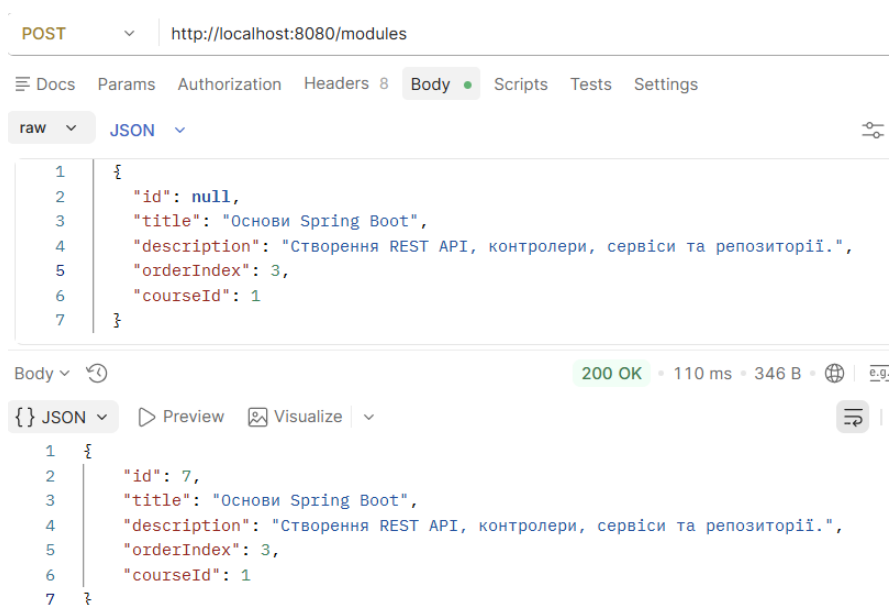


Рисунок 4.25 – Результат тестування створення навчального модуля

```

21  {
22  },
23  {
24      "id": 4,
25      "title": "Python Basics",
26      "description": "Python syntax and first programs",
27      "orderIndex": 1,
28      "courseId": 3
29  },
30  {
31      "id": 5,
32      "title": "Object Oriented Programming in C++",
33      "description": "Classes, objects and inheritance in C++",
34      "orderIndex": 1,
35      "courseId": 4
36  },
37  {
38      "id": 6,
39      "title": "JavaScript DOM Manipulation",
40      "description": "Working with HTML elements using JavaScript",
41      "orderIndex": 1,
42      "courseId": 5
43  },
44  {
45      "id": 7,
46      "title": "Основи Spring Boot",
47      "description": "Створення REST API, контролери, сервіси та репозиторії.",
48      "orderIndex": 3,
49      "courseId": 1
50  }
51  ]

```

Рисунок 4.26 – Результат тестування отримання списку навчальних модулів

```

1  {
2      "id": 3,
3      "title": "Collections Framework",
4      "description": "Understanding lists, sets and maps",
5      "orderIndex": 1,
6      "courseId": 2
7  }

```

Рисунок 4.27 – Результат тестування отримання навчального модуля за id



Рисунок 4.28 – Результат тестування оновлення навчального модуля

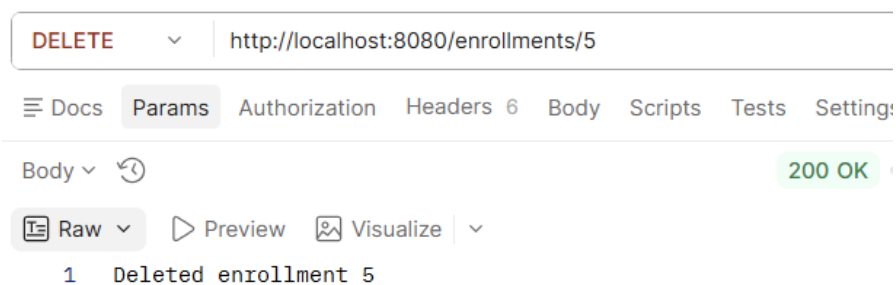


Рисунок 4.29 – Результат тестування видалення навчального модуля

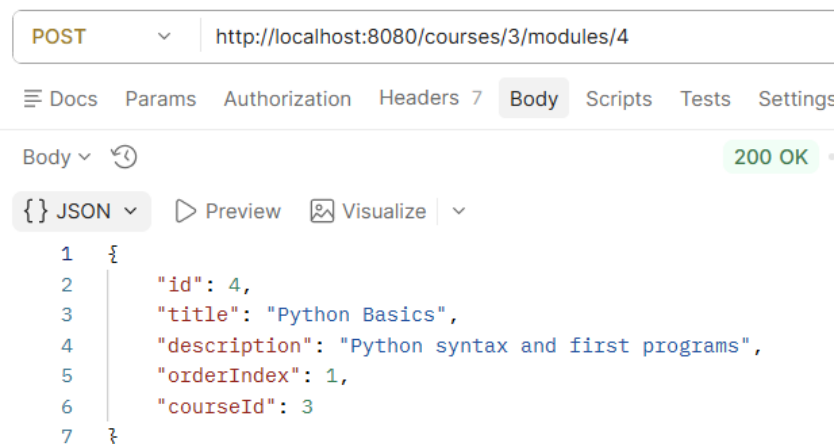


Рисунок 4.30 – Результат тестування додавання модуля до курсу

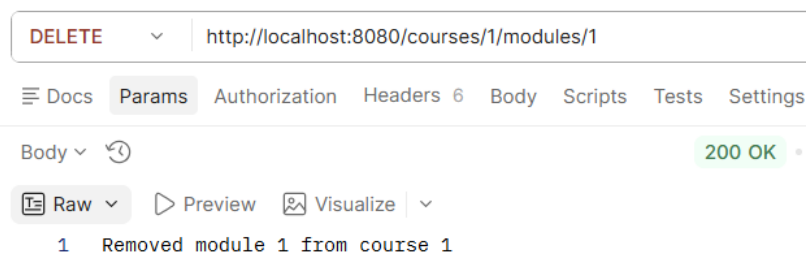


Рисунок 4.31 – Результат тестування видалення модуля з курсу

У результаті тестування було підтверджено, що модулі створюються, переглядаються, оновлюються, видаляються та можуть бути прив'язані до конкретного курсу.

4.6 Тестування пошуку та аналітичних запитів

Пошук і аналітичні запити перевірялися через endpoint'и /search та /analytics. Для пошуку використовувалися запити GET /search/courses?query=value, GET /search/courses/advanced?query=value&level=value та GET /search/students?query=value. Результати тестування пошуку наведено на рисунках 4.32–4.34.



Рисунок 4.32 – Результат тестування пошуку курсів

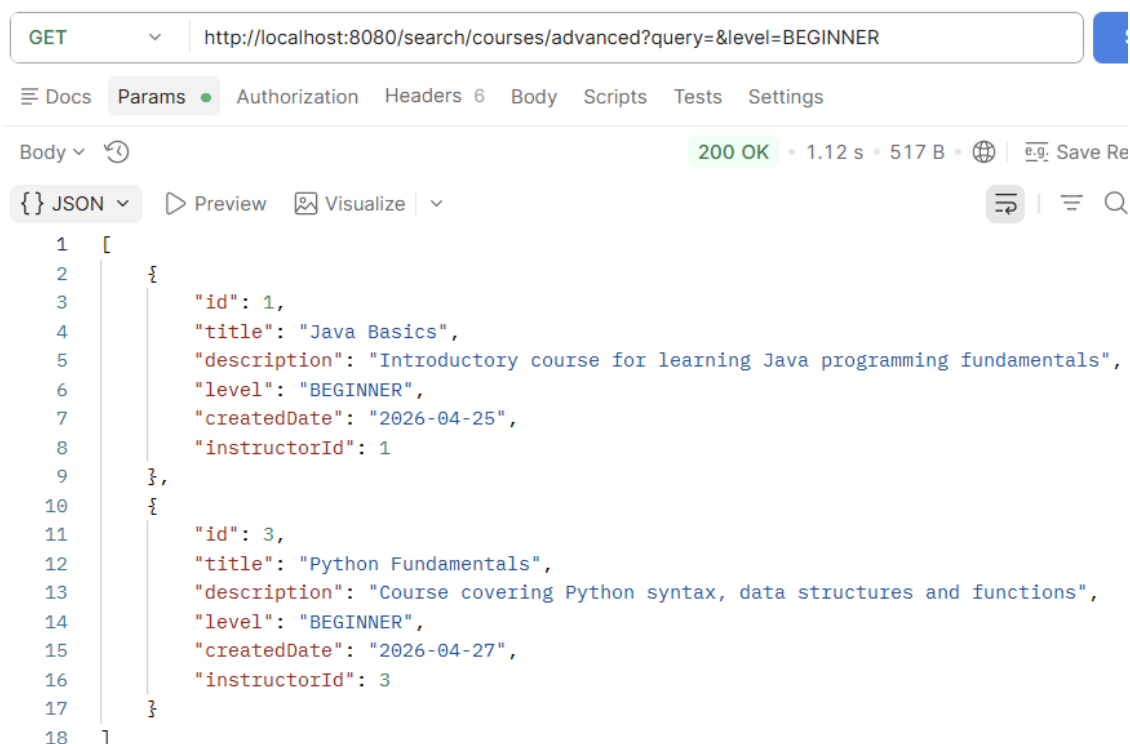


Рисунок 4.33 – Результат тестування розширеного пошуку курсів

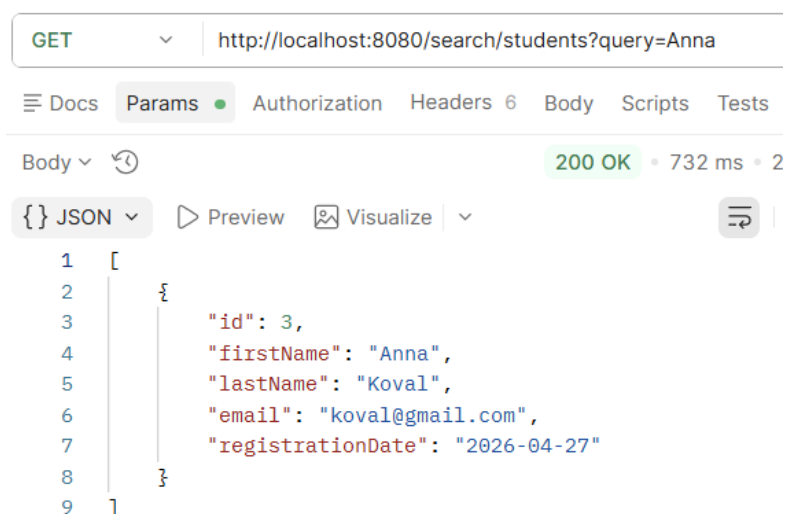


Рисунок 4.34 – Результат тестування пошуку студентів

Аналітичні endpoint'и перевірялися через запити GET /analytics/courses/count, GET /analytics/students/count, GET /analytics/enrollments/active, GET /analytics/enrollments/completed та GET /analytics/courses/by-level. Результати тестування аналітичних запитів наведено на рисунках 4.35–4.39.

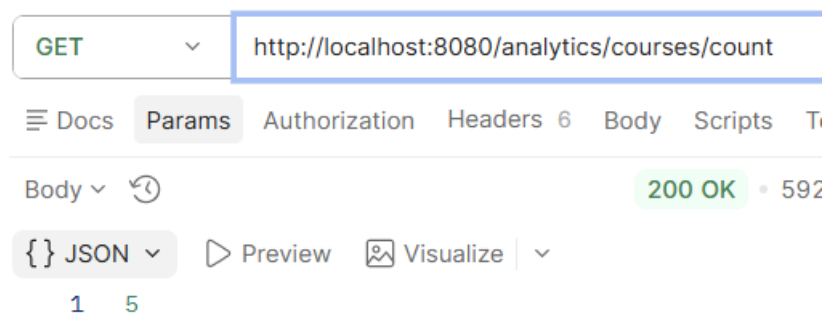


Рисунок 4.35 – Результат тестування підрахунку кількості курсів

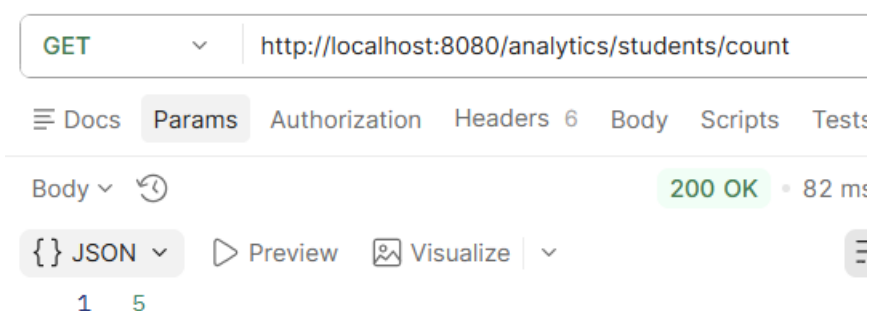


Рисунок 4.36 – Результат тестування підрахунку кількості студентів

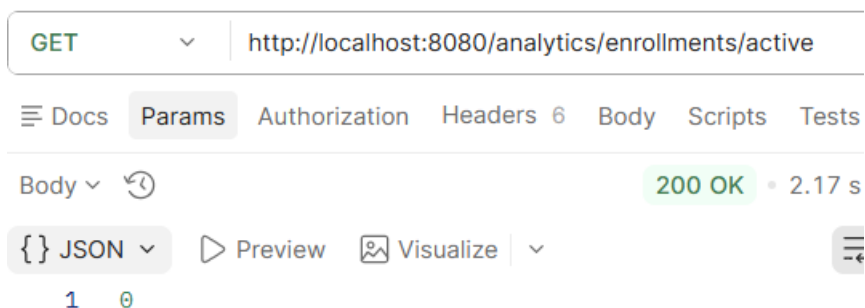


Рисунок 4.37 – Результат тестування підрахунку активних записів

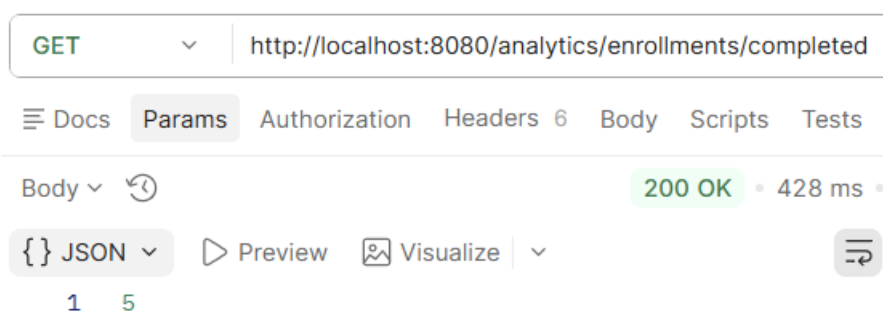


Рисунок 4.38 – Результат тестування підрахунку завершених записів

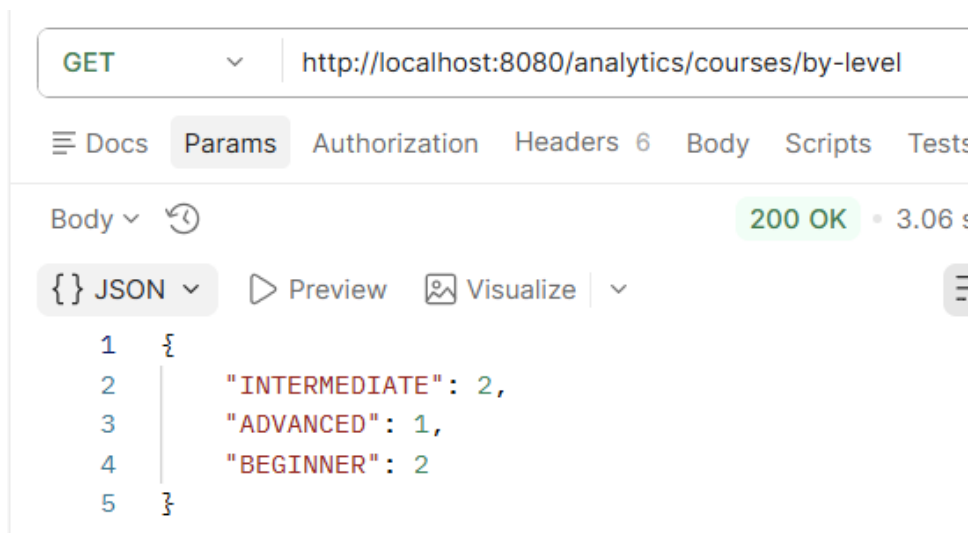


Рисунок 4.39 – Результат тестування підрахунку курсів за рівнями складності

У результаті тестування було встановлено, що пошук повертає потрібні записи, а аналітичні endpoint'и коректно формують статистичну інформацію.

4.7 Перевірка працездатності сервісу через actuator

Для перевірки працездатності backend-сервісу використовувалися endpoint'и GET /actuator/health, GET /actuator/metrics та GET /actuator/prometheus. Результати перевірки наведено на рисунках 4.40–4.42.

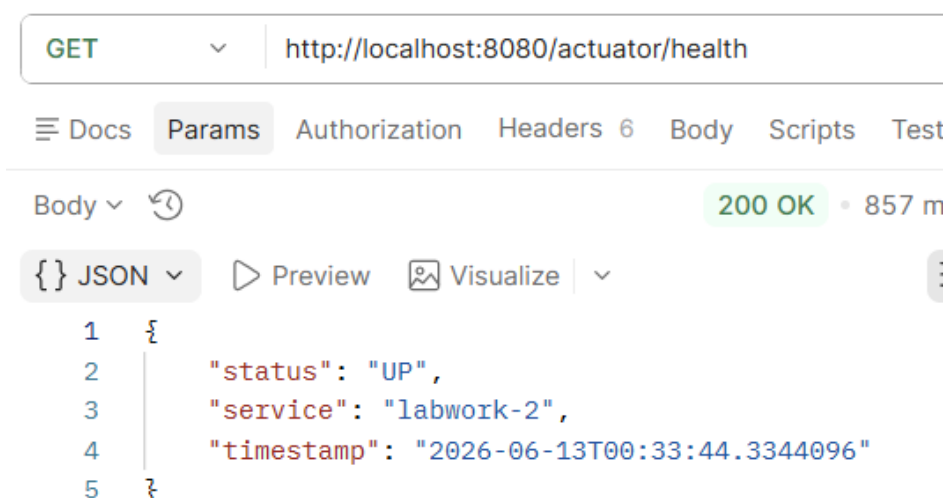


Рисунок 4.40 – Результат перевірки працездатності сервісу через actuator

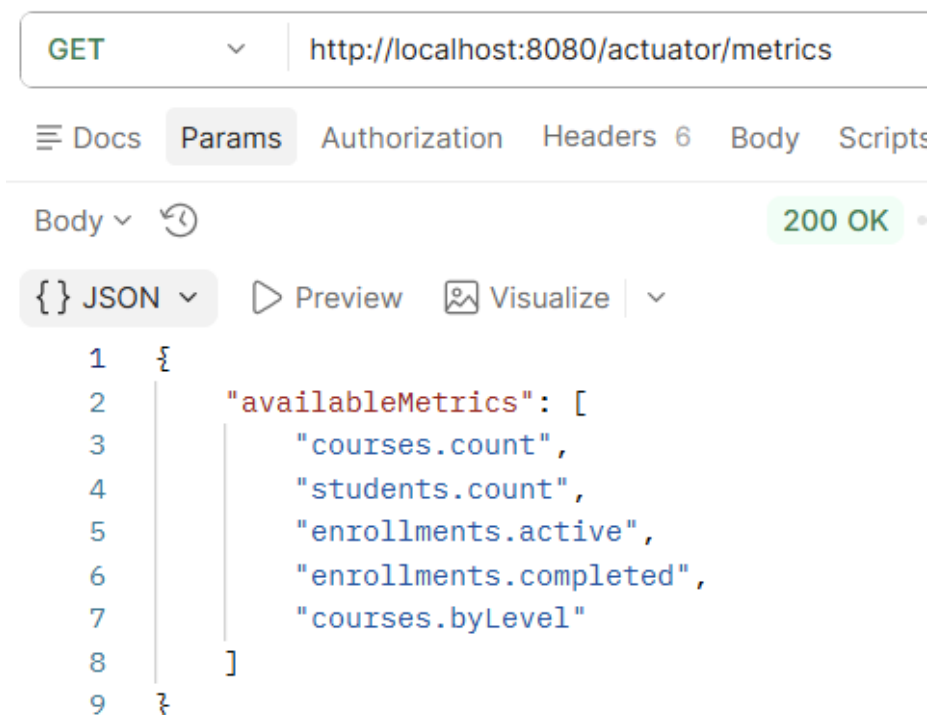


Рисунок 4.41 – Результат отримання метрик backend-сервісу

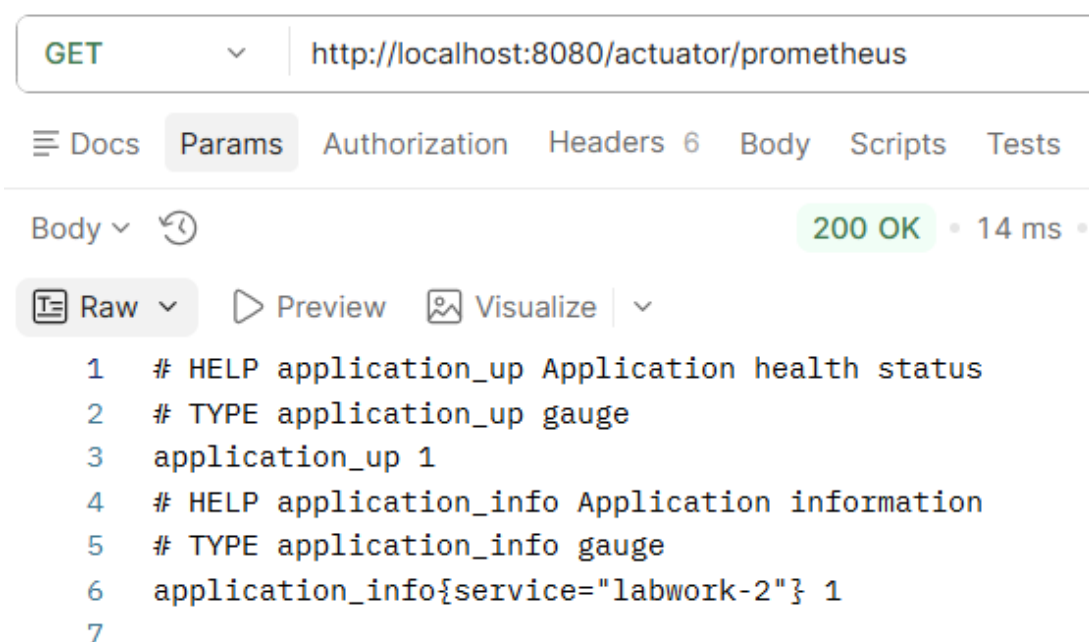


Рисунок 4.42 – Результат отримання метрик у форматі Prometheus

Після перевірки цих endpoint'ів було зроблено висновок, що сервіс запускається, відповідає на запити та надає інформацію про свій стан.

Висновки до розділу 4

У четвертому розділі було описано тестування backend-сервісу управління онлайн-курсами. Було перевірено API для роботи з викладачами, курсами, студентами, записами студентів на курси та навчальними модулями.

Також було протестовано пошук курсів і студентів, аналітичні endpoint'и та перевірку працездатності сервісу через actuator. Для кожного виду тестування наведено відповідні рисунки з результатами виконання HTTP-запитів. У результаті тестування підтверджено, що основні функції backend-сервісу працюють коректно та повертають очікувані відповіді.

ЗАГАЛЬНІ ВИСНОВКИ

У ході виконання курсової роботи було розроблено backend-сервіс для управління онлайн-курсами. У роботі було розглянуто предметну область онлайн-навчання, визначено основні вимоги до системи та описано її основних користувачів.

Було спроектовано структуру програмного забезпечення, визначено основні сутності системи: викладач, курс, студент, запис на курс та навчальний модуль. Також було описано зв'язки між сутностями, структуру бази даних, REST API, пошук, аналітичні запити та перевірку стану сервісу.

Під час реалізації програмного продукту було використано Java, Spring Boot, Spring Data JPA, Hibernate ORM та PostgreSQL. У системі реалізовано роботу з викладачами, курсами, студентами, навчальними модулями та записами студентів на курси. Також було додано DTO-класи, валідацію даних, централізовану обробку помилок, пошук, аналітичні endpoint'и та endpoint'и для перевірки працездатності сервісу.

У процесі тестування було перевірено основні функції backend-сервісу через REST API. Було протестовано створення, отримання, оновлення та видалення даних, запис студентів на курси, пошук, аналітичні запити та перевірку стану сервісу. Результати тестування підтвердили, що розроблений backend-сервіс працює коректно та виконує поставлені завдання.

Отже, мету курсової роботи було досягнуто. Розроблений програмний продукт забезпечує базову функціональність системи управління онлайн-курсами та може бути використаний як основа для подальшого розширення, наприклад додавання авторизації користувачів, ролей, оцінювання студентів та розширеної статистики навчання.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Oracle Corporation. Java Documentation. URL: <https://docs.oracle.com/en/java/> (дата звернення: 12.06.2026).
2. Spring Boot Documentation. URL: <https://docs.spring.io/spring-boot/documentation.html> (дата звернення: 12.06.2026).
3. Spring Boot Reference Documentation. URL: <https://docs.spring.io/spring-boot/reference/> (дата звернення: 12.06.2026).
4. Spring Boot Project. URL: <https://spring.io/projects/spring-boot> (дата звернення: 12.06.2026).
5. Spring Framework Documentation. URL: <https://docs.spring.io/spring-framework/reference/> (дата звернення: 12.06.2026).
6. Spring Data JPA. URL: <https://spring.io/projects/spring-data-jpa> (дата звернення: 12.06.2026).
7. Hibernate ORM Documentation. URL: <https://hibernate.org/orm/documentation/>
8. PostgreSQL Documentation. URL: <https://www.postgresql.org/docs/current/> (дата звернення: 12.06.2026).
9. PostgreSQL About. URL: <https://www.postgresql.org/about/> (дата звернення: 12.06.2026).
10. PostgreSQL Tutorial. URL: <https://www.postgresql.org/docs/current/tutorial.html> (дата звернення: 12.06.2026).
11. OpenAPI Specification. URL: <https://swagger.io/specification/>
12. SpringDoc OpenAPI. URL: <https://springdoc.org/>
13. Spring Boot Actuator Documentation. URL: <https://docs.spring.io/spring-boot/reference/actuator/>

ДОДАТОК А

Вихідний код сутностей системи

У додатку А наведено програмний код сутностей backend-сервісу управління онлайн-курсами. Дані класи описують основні об'єкти предметної області та використовуються для формування таблиць бази даних.

Лістинг А.1 – Код класу Instructor

```
import jakarta.persistence.*;
import ua.opnu.labwork2.course.model.Course;
import java.util.List;

@Entity
public class Instructor {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String firstName;
    private String lastName;
    private String email;
    private String specialization;

    @OneToMany(mappedBy = "instructor")
    private List<Course> courses;

    public Instructor() {
    }

    public Instructor(Long id, String firstName, String lastName, String email,
String specialization) {
        this.id = id;

        this.firstName = firstName;
        this.lastName = lastName;
        this.email = email;
        this.specialization = specialization;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getFirstName() {
        return firstName;
    }
}
```

```

    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }

    public String getLastName() {
        return lastName;
    }

    public void setLastName(String lastName) {
        this.lastName = lastName;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getSpecialization() {
        return specialization;
    }

    public void setSpecialization(String specialization) {
        this.specialization = specialization;
    }
}

```

Лістинг А.2 – Код класу Course

```

import jakarta.persistence.*;
import ua.opnu.labwork2.enrollment.model.Enrollment;
import ua.opnu.labwork2.instructor.model.Instructor;
import ua.opnu.labwork2.module.model.CourseModule;

import java.time.LocalDate;
import java.util.List;

@Entity
public class Course {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    @Column(columnDefinition = "TEXT")
    private String description;
    private String level;
    private LocalDate createdAt;

    @ManyToOne
    @JoinColumn(name = "instructor_id")
    private Instructor instructor;

    @OneToMany(mappedBy = "course")
    private List<CourseModule> modules;

    @OneToMany(mappedBy = "course")

```

```

private List<Enrollment> enrollments;

public Course() {
}

public Course(Long id, String title, String description, String level,
LocalDate createdAt) {
    this.id = id;
    this.title = title;
    this.description = description;
    this.level = level;
    this.createdAt = createdAt;
}

public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getTitle() {
    return title;
}

public void setTitle(String title) {
    this.title = title;
}

public String getDescription() {
    return description;
}

public void setDescription(String description) {
    this.description = description;
}

public String getLevel() {
    return level;
}

public void setLevel(String level) {
    this.level = level;
}

public LocalDate getCreatedAt() {
    return createdAt;
}

public void setCreatedAt(LocalDate createdAt) {
    this.createdAt = createdAt;
}

public Instructor getInstructor() {
    return instructor;
}

public void setInstructor(Instructor instructor) {
    this.instructor = instructor;
}
}

```

ЛІСТИНГ А.3 – Код класу Student

```
import jakarta.persistence.*;
import ua.opnu.labwork2.enrollment.model.Enrollment;
import java.time.LocalDate;
import java.util.List;

@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String firstName;
    private String lastName;
    private String email;
    private LocalDate registrationDate;

    @OneToMany(mappedBy = "student")
    private List<Enrollment> enrollments;

    public Student() {
    }

    public Student(Long id, String firstName, String lastName, String email,
LocalDate registrationDate) {
        this.id = id;
        this.firstName = firstName;
        this.lastName = lastName;
        this.email = email;
        this.registrationDate = registrationDate;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getFirstName() {
        return firstName;
    }

    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }

    public String getLastName() {
        return lastName;
    }

    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
}
```

```

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public LocalDate getRegistrationDate() {
    return registrationDate;
}

public void setRegistrationDate(LocalDate registrationDate) {
    this.registrationDate = registrationDate;
}
}

```

ЛІСТИНГ А.4 – Код класу Enrollment

```

import jakarta.persistence.*;
import ua.opnu.labwork2.course.model.Course;
import ua.opnu.labwork2.student.model.Student;
import java.time.LocalDate;

@Entity
public class Enrollment {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private LocalDate enrollmentDate;
    private LocalDate completionDate;
    private Integer progress;

    @ManyToOne
    @JoinColumn(name = "student_id")
    private Student student;

    @ManyToOne
    @JoinColumn(name = "course_id")
    private Course course;

    public Enrollment() {
    }

    public Enrollment(Long id, LocalDate enrollmentDate, LocalDate completionDate,
Integer progress) {
        this.id = id;

        this.enrollmentDate = enrollmentDate;
        this.completionDate = completionDate;
        this.progress = progress;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {

```

```

        this.id = id;
    }

    public LocalDate getEnrollmentDate() {
        return enrollmentDate;
    }

    public LocalDate getCompletionDate() {
        return completionDate;
    }

    public Integer getProgress() {
        return progress;
    }

    public Course getCourse() {
        return course;
    }

    public void setCourse(Course course) {
        this.course = course;
    }

    public Student getStudent() {
        return student;
    }

    public void setStudent(Student student) {
        this.student = student;
    }

    public void setProgress(Integer progress) {
        this.progress = progress;
    }

    public void setCompletionDate(LocalDate completionDate) {
        this.completionDate = completionDate;
    }

    public void setEnrollmentDate(LocalDate enrollmentDate) {
        this.enrollmentDate = enrollmentDate;
    }
}

```

Лістинг А.5 – Код класу CourseModule

```

import jakarta.persistence.*;
import ua.opnu.labwork2.course.model.Course;

@Entity
public class CourseModule {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    @Column(columnDefinition = "TEXT")
    private String description;
    private Integer orderIndex;
}

```

```
@ManyToOne
@JoinColumn(name = "course_id")
private Course course;

public CourseModule() {
}

public CourseModule(Long id, String title, String description, Integer
orderIndex) {
    this.id = id;
    this.title = title;
    this.description = description;
    this.orderIndex = orderIndex;
}

public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getTitle() {
    return title;
}

public void setTitle(String title) {
    this.title = title;
}

public String getDescription() {
    return description;
}

public void setDescription(String description) {
    this.description = description;
}

public Integer getOrderIndex() {
    return orderIndex;
}

public void setOrderIndex(Integer orderIndex) {
    this.orderIndex = orderIndex;
}

public Course getCourse() {
    return course;
}

public void setCourse(Course course) {
    this.course = course;
}
}
```

ДОДАТОК Б

Вихідний код dto-класів

У додатку Б наведено DTO-класи, які використовуються для передачі даних між клієнтом і backend-сервісом. Окремі DTO застосовуються для створення, оновлення та повернення даних у відповідях API.

Лістинг Б.1 – Код класу InstructorCreateRequest

```
import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

@Schema(description = "Дані для створення викладача")
public record InstructorCreateRequest(
    @Schema(description = "Ім'я викладача", example = "Марія", requiredMode =
Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "First name is required")
    @Size(min = 2, max = 50, message = "First name must contain from 2 to 50
characters")
    String firstName,

    @Schema(description = "Прізвище викладача", example = "Іваненко",
requiredMode = Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "Last name is required")
    @Size(min = 2, max = 50, message = "Last name must contain from 2 to 50
characters")
    String lastName,

    @Schema(description = "Унікальний email викладача", example =
"m.ivanenko@op.edu.ua", requiredMode = Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "Email is required")
    @Email(message = "Email format is invalid")
    @Size(max = 100, message = "Email cannot be longer than 100 characters")
    String email,

    @Schema(description = "Спеціалізація викладача", example = "Java та Spring
Boot", requiredMode = Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "Specialization is required")
    @Size(min = 3, max = 100, message = "Specialization must contain from 3 to
100 characters")
    String specialization
) {
}
```

Лістинг Б.2 – Код класу InstructorUpdateRequest

```
import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
```

```

import jakarta.validation.constraints.Size;

@Schema(description = "Дані для оновлення викладача")
public record InstructorUpdateRequest(
    @Schema(description = "Ім'я викладача", example = "Андрій", requiredMode =
Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "First name is required")
    @Size(min = 2, max = 50, message = "First name must contain from 2 to 50
characters")
    String firstName,

    @Schema(description = "Прізвище викладача", example = "Шевченко",
requiredMode = Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "Last name is required")
    @Size(min = 2, max = 50, message = "Last name must contain from 2 to 50
characters")
    String lastName,

    @Schema(description = "Унікальний email викладача", example =
"a.shevchenko@op.edu.ua", requiredMode = Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "Email is required")
    @Email(message = "Email format is invalid")
    @Size(max = 100, message = "Email cannot be longer than 100 characters")
    String email,

    @Schema(description = "Спеціалізація викладача", example = "Backend
розробка", requiredMode = Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "Specialization is required")
    @Size(min = 3, max = 100, message = "Specialization must contain from 3 to
100 characters")
    String specialization
) {
}

```

Лістинг Б.3 – Код класу InstructorResponse

```

import io.swagger.v3.oas.annotations.media.Schema;

@Schema(description = "Вихідна модель викладача")
public record InstructorResponse(
    @Schema(description = "Ідентифікатор викладача", example = "1")
    Long id,
    @Schema(description = "Ім'я викладача", example = "Марія")
    String firstName,
    @Schema(description = "Прізвище викладача", example = "Іваненко")
    String lastName,
    @Schema(description = "Email викладача", example = "m.ivanenko@op.edu.ua")
    String email,
    @Schema(description = "Спеціалізація викладача", example = "Java та Spring
Boot")
    String specialization
) {
}

```

Лістинг Б.4 – Код класу CourseCreateRequest

```

import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.PastOrPresent;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Positive;
import jakarta.validation.constraints.Size;

import java.time.LocalDate;

@Schema(description = "Дані для створення нового онлайн-курсу")
public record CourseCreateRequest(
    @Schema(description = "Назва курсу", example = "Java для початківців",
requiredMode = Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "Course title is required")
    @Size(min = 3, max = 150, message = "Course title must contain from 3 to
150 characters")
    String title,

    @Schema(description = "Опис змісту курсу", example = "Повний вступний курс
з Java, ООП, колекціями та базовими принципами Spring.", requiredMode =
Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "Course description is required")
    @Size(min = 20, max = 2000, message = "Course description must contain
from 20 to 2000 characters")
    String description,

    @Schema(description = "Рівень складності курсу", example = "BEGINNER",
allowableValues = {"BEGINNER", "INTERMEDIATE", "ADVANCED"}, requiredMode =
Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "Course level is required")
    @Pattern(regexp = "BEGINNER|INTERMEDIATE|ADVANCED", message = "Course
level must be BEGINNER, INTERMEDIATE or ADVANCED")
    String level,

    @Schema(description = "Дата створення курсу", example = "2026-05-10",
requiredMode = Schema.RequiredMode.REQUIRED)
    @NotNull(message = "Created date is required")
    @PastOrPresent(message = "Created date cannot be in the future")
    LocalDate createdAt,

    @Schema(description = "Ідентифікатор викладача, який веде курс", example =
"1", requiredMode = Schema.RequiredMode.REQUIRED)
    @NotNull(message = "Instructor id is required")
    @Positive(message = "Instructor id must be positive")
    Long instructorId
) {
}

```

Лістинг Б.5 – Код класу CourseUpdateRequest

```

import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.PastOrPresent;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Positive;
import jakarta.validation.constraints.Size;

```

```

import java.time.LocalDate;

@Schema(description = "Дані для оновлення онлайн-курсу")
public record CourseUpdateRequest(
    @Schema(description = "Оновлена назва курсу", example = "Java Core та Spring", requiredMode = Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "Course title is required")
    @Size(min = 3, max = 150, message = "Course title must contain from 3 to 150 characters")
    String title,

    @Schema(description = "Оновлений опис курсу", example = "Курс охоплює Java Core, принципи ООП, JPA та створення REST API.", requiredMode = Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "Course description is required")
    @Size(min = 20, max = 2000, message = "Course description must contain from 20 to 2000 characters")
    String description,

    @Schema(description = "Рівень складності курсу", example = "INTERMEDIATE", allowableValues = {"BEGINNER", "INTERMEDIATE", "ADVANCED"}, requiredMode = Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "Course level is required")
    @Pattern(regexp = "BEGINNER|INTERMEDIATE|ADVANCED", message = "Course level must be BEGINNER, INTERMEDIATE or ADVANCED")
    String level,

    @Schema(description = "Дата створення або оновлення запису курсу", example = "2026-05-10", requiredMode = Schema.RequiredMode.REQUIRED)
    @NotNull(message = "Created date is required")
    @PastOrPresent(message = "Created date cannot be in the future")
    LocalDate createdAt,

    @Schema(description = "Ідентифікатор викладача курсу", example = "2", requiredMode = Schema.RequiredMode.REQUIRED)
    @NotNull(message = "Instructor id is required")
    @Positive(message = "Instructor id must be positive")
    Long instructorId
) {
}

```

Лістинг Б.6 – Код класу CourseResponse

```

import io.swagger.v3.oas.annotations.media.Schema;
import java.time.LocalDate;

@Schema(description = "Вихідна модель курсу")
public record CourseResponse(
    @Schema(description = "Ідентифікатор курсу", example = "1")
    Long id,
    @Schema(description = "Назва курсу", example = "Java для початківців")
    String title,
    @Schema(description = "Опис курсу", example = "Повний вступний курс з Java та Spring.")
    String description,
    @Schema(description = "Рівень складності", example = "BEGINNER")
    String level,
    @Schema(description = "Дата створення", example = "2026-05-10")
    LocalDate createdAt,

```

```

        @Schema(description = "Ідентифікатор викладача", example = "1")
        Long instructorId
    ) {
    }

```

Лістинг Б.7 – Код класу StudentCreateRequest

```

import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.PastOrPresent;
import jakarta.validation.constraints.Size;

import java.time.LocalDate;

@Schema(description = "Дані для реєстрації студента")
public record StudentCreateRequest(
    @Schema(description = "Ім'я студента", example = "Іван", requiredMode =
Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "First name is required")
    @Size(min = 2, max = 50, message = "First name must contain from 2 to 50
characters")
    String firstName,

    @Schema(description = "Прізвище студента", example = "Петренко",
requiredMode = Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "Last name is required")
    @Size(min = 2, max = 50, message = "Last name must contain from 2 to 50
characters")
    String lastName,

    @Schema(description = "Email студента, використовується як унікальний
контакт", example = "ivan.petrenko@op.edu.ua", requiredMode =
Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "Email is required")
    @Email(message = "Email format is invalid")
    @Size(max = 100, message = "Email cannot be longer than 100 characters")
    String email,

    @Schema(description = "Дата реєстрації студента", example = "2026-05-10",
requiredMode = Schema.RequiredMode.REQUIRED)
    @NotNull(message = "Registration date is required")
    @PastOrPresent(message = "Registration date cannot be in the future")
    LocalDate registrationDate
) {
}

```

Лістинг Б.8 – Код класу StudentUpdateRequest

```

import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.PastOrPresent;
import jakarta.validation.constraints.Size;

```

```

import java.time.LocalDate;

@Schema(description = "Дані для оновлення студента")
public record StudentUpdateRequest(
    @Schema(description = "Ім'я студента", example = "Олена", requiredMode =
Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "First name is required")
    @Size(min = 2, max = 50, message = "First name must contain from 2 to 50
characters")
    String firstName,

    @Schema(description = "Прізвище студента", example = "Коваль",
requiredMode = Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "Last name is required")
    @Size(min = 2, max = 50, message = "Last name must contain from 2 to 50
characters")
    String lastName,

    @Schema(description = "Email студента", example = "olena.koval@op.edu.ua",
requiredMode = Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "Email is required")
    @Email(message = "Email format is invalid")
    @Size(max = 100, message = "Email cannot be longer than 100 characters")
    String email,

    @Schema(description = "Дата реєстрації студента", example = "2026-05-10",
requiredMode = Schema.RequiredMode.REQUIRED)
    @NotNull(message = "Registration date is required")
    @PastOrPresent(message = "Registration date cannot be in the future")
    LocalDate registrationDate
) {
}

```

Лістинг Б.9 – Код класу StudentResponse

```

import io.swagger.v3.oas.annotations.media.Schema;

import java.time.LocalDate;

@Schema(description = "Вихідна модель студента")
public record StudentResponse(
    @Schema(description = "Ідентифікатор студента", example = "1")
    Long id,
    @Schema(description = "Ім'я студента", example = "Іван")
    String firstName,
    @Schema(description = "Прізвище студента", example = "Петренко")
    String lastName,
    @Schema(description = "Email студента", example =
"ivan.petrenko@op.edu.ua")
    String email,
    @Schema(description = "Дата реєстрації", example = "2026-05-10")
    LocalDate registrationDate
) {
}

```

Лістинг Б.10 – Код класу EnrollmentCreateRequest

```

import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.PastOrPresent;
import jakarta.validation.constraints.Positive;

import java.time.LocalDate;

@Schema(description = "Дані для створення запису студента на курс")
public record EnrollmentCreateRequest(
    @Schema(description = "Ідентифікатор студента", example = "1",
        requiredMode = Schema.RequiredMode.REQUIRED)
        @NotNull(message = "Student id is required")
        @Positive(message = "Student id must be positive")
        Long studentId,

    @Schema(description = "Ідентифікатор курсу", example = "3", requiredMode =
        Schema.RequiredMode.REQUIRED)
        @NotNull(message = "Course id is required")
        @Positive(message = "Course id must be positive")
        Long courseId,

    @Schema(description = "Дата запису студента на курс", example = "2026-05-
10", requiredMode = Schema.RequiredMode.REQUIRED)
        @NotNull(message = "Enrollment date is required")
        @PastOrPresent(message = "Enrollment date cannot be in the future")
        LocalDate enrollmentDate,

    @Schema(description = "Дата завершення курсу студентом", example = "2026-
06-20")
        LocalDate completionDate,

    @Schema(description = "Поточний прогрес проходження курсу у відсотках",
        example = "25", requiredMode = Schema.RequiredMode.REQUIRED)
        @NotNull(message = "Progress is required")
        @Min(value = 0, message = "Progress cannot be less than 0")
        @Max(value = 100, message = "Progress cannot be greater than 100")
        Integer progress
    ) {
}

```

Лістинг Б.11 – Код класу EnrollmentUpdateRequest

```

import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.PastOrPresent;
import jakarta.validation.constraints.Positive;

import java.time.LocalDate;

@Schema(description = "Дані для повного оновлення запису студента на курс")
public record EnrollmentUpdateRequest(
    @Schema(description = "Ідентифікатор студента", example = "1",
        requiredMode = Schema.RequiredMode.REQUIRED)
        @NotNull(message = "Student id is required")
        @Positive(message = "Student id must be positive")

```

```

        Long studentId,

        @Schema(description = "Ідентифікатор курсу", example = "3", requiredMode =
Schema.RequiredMode.REQUIRED)
        @NotNull(message = "Course id is required")
        @Positive(message = "Course id must be positive")
        Long courseId,

        @Schema(description = "Дата запису студента на курс", example = "2026-05-
10", requiredMode = Schema.RequiredMode.REQUIRED)
        @NotNull(message = "Enrollment date is required")
        @PastOrPresent(message = "Enrollment date cannot be in the future")
        LocalDate enrollmentDate,

        @Schema(description = "Дата завершення курсу студентом", example = "2026-
06-20")
        LocalDate completionDate,

        @Schema(description = "Поточний прогрес проходження курсу у відсотках",
example = "70", requiredMode = Schema.RequiredMode.REQUIRED)
        @NotNull(message = "Progress is required")
        @Min(value = 0, message = "Progress cannot be less than 0")
        @Max(value = 100, message = "Progress cannot be greater than 100")
        Integer progress
    ) {
    }
}

```

Лістинг Б.12 – Код класу EnrollmentProgressUpdateRequest

```

import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotNull;

@Schema(description = "Дані для оновлення прогресу проходження курсу")
public record EnrollmentProgressUpdateRequest(
    @Schema(description = "Поточний прогрес проходження курсу у відсотках",
example = "85", requiredMode = Schema.RequiredMode.REQUIRED)
    @NotNull(message = "Progress is required")
    @Min(value = 0, message = "Progress cannot be less than 0")
    @Max(value = 100, message = "Progress cannot be greater than 100")
    Integer progress
) {
}

```

Лістинг Б.13 – Код класу EnrollmentResponse

```

import io.swagger.v3.oas.annotations.media.Schema;

import java.time.LocalDate;

@Schema(description = "Вихідна модель запису студента на курс")
public record EnrollmentResponse(
    @Schema(description = "Ідентифікатор запису на курс", example = "1")
    Long id,
    @Schema(description = "Ідентифікатор студента", example = "1")

```



```

        Long studentId,
        @Schema(description = "Ідентифікатор курсу", example = "3")
        Long courseId,
        @Schema(description = "Дата запису студента на курс", example = "2026-05-
10")
        LocalDate enrollmentDate,
        @Schema(description = "Дата завершення курсу студентом", example = "2026-
06-20")
        LocalDate completionDate,
        @Schema(description = "Поточний прогрес проходження курсу у відсотках",
example = "85")
        Integer progress
    ) {
    }
}

```

Лістинг Б.14 – Код класу ModuleCreateRequest

```

import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;
import jakarta.validation.constraints.Size;

@Schema(description = "Дані для створення навчального модуля курсу")
public record ModuleCreateRequest(
    @Schema(description = "Назва модуля", example = "Вступ до Java",
requiredMode = Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "Module title is required")
    @Size(min = 3, max = 150, message = "Module title must contain from 3 to
150 characters")
    String title,

    @Schema(description = "Опис тем і матеріалів модуля", example = "Базові
поняття мови Java, типи даних та перша програма.", requiredMode =
Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "Module description is required")
    @Size(min = 10, max = 1000, message = "Module description must contain
from 10 to 1000 characters")
    String description,

    @Schema(description = "Порядковий номер модуля в курсі", example = "1",
requiredMode = Schema.RequiredMode.REQUIRED)
    @NotNull(message = "Order index is required")
    @Positive(message = "Order index must be positive")
    Integer orderIndex,

    @Schema(description = "Ідентифікатор курсу, до якого належить модуль",
example = "3", requiredMode = Schema.RequiredMode.REQUIRED)
    @NotNull(message = "Course id is required")
    @Positive(message = "Course id must be positive")
    Long courseId
) {
}

```

Лістинг Б.15 – Код класу ModuleUpdateRequest

```

import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;
import jakarta.validation.constraints.Size;

@Schema(description = "Дані для оновлення навчального модуля курсу")
public record ModuleUpdateRequest(
    @Schema(description = "Назва модуля", example = "Основи Spring Boot",
requiredMode = Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "Module title is required")
    @Size(min = 3, max = 150, message = "Module title must contain from 3 to
150 characters")
    String title,

    @Schema(description = "Опис тем і матеріалів модуля", example = "Створення
REST API, контролери, сервіси та репозиторії.", requiredMode =
Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "Module description is required")
    @Size(min = 10, max = 1000, message = "Module description must contain
from 10 to 1000 characters")
    String description,

    @Schema(description = "Порядковий номер модуля в курсі", example = "2",
requiredMode = Schema.RequiredMode.REQUIRED)
    @NotNull(message = "Order index is required")
    @Positive(message = "Order index must be positive")
    Integer orderIndex,

    @Schema(description = "Ідентифікатор курсу, до якого належить модуль",
example = "3", requiredMode = Schema.RequiredMode.REQUIRED)
    @NotNull(message = "Course id is required")
    @Positive(message = "Course id must be positive")
    Long courseId
) {
}

```

Лістинг Б.16 – Код класу ModuleResponse

```

import io.swagger.v3.oas.annotations.media.Schema;

@Schema(description = "Вихідна модель навчального модуля курсу")
public record ModuleResponse(
    @Schema(description = "Ідентифікатор модуля", example = "1")
    Long id,
    @Schema(description = "Назва модуля", example = "Вступ до Java")
    String title,
    @Schema(description = "Опис тем і матеріалів модуля", example = "Базові
поняття мови Java, типи даних та перша програма.")
    String description,
    @Schema(description = "Порядковий номер модуля в курсі", example = "1")
    Integer orderIndex,
    @Schema(description = "Ідентифікатор курсу, до якого належить модуль",
example = "3")
    Long courseId
) {
}

```

ДОДАТОК В

Вихідний код репозиторіїв

У додатку В наведено репозиторії, які забезпечують доступ до бази даних. Вони використовуються для виконання операцій створення, отримання, оновлення, видалення, пошуку та аналітичних запитів.

Лістинг В.1 – Код інтерфейсу InstructorRepository

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import ua.opnu.labwork2.instructor.model.Instructor;

@Repository
public interface InstructorRepository extends JpaRepository<Instructor, Long> {
    boolean existsByEmail(String email);

    boolean existsByEmailAndIdNot(String email, Long id);
}
```

Лістинг В.2 – Код інтерфейсу CourseRepository

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;
import ua.opnu.labwork2.course.model.Course;

import java.util.List;

@Repository
public interface CourseRepository extends JpaRepository<Course, Long> {
    List<Course>
    findByTitleContainingIgnoreCaseOrDescriptionContainingIgnoreCase(String title,
    String description);

    @Query("""
        select c from Course c
        where (:level is null or lower(c.level) = lower(:level))
            and (lower(c.title) like lower(concat('%', :query, '%'))
                or lower(c.description) like lower(concat('%', :query, '%')))
        """)
    List<Course> searchByQueryAndLevel(@Param("query") String query,
    @Param("level") String level);

    @Query("select c.level, count(c) from Course c group by c.level")
    List<Object[]> countCoursesByLevel();
}
```

Лістинг В.3 – Код інтерфейсу StudentRepository

```

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import ua.opnu.labwork2.student.model.Student;

import java.util.List;

@Repository
public interface StudentRepository extends JpaRepository<Student, Long> {
    boolean existsByEmail(String email);

    boolean existsByEmailAndIdNot(String email, Long id);

    List<Student>
    findByFirstNameContainingIgnoreCaseOrLastNameContainingIgnoreCaseOrEmailContaining
    IgnoreCase(
        String firstName,
        String lastName,
        String email
    );
}

```

Лістинг В.4 – Код інтерфейсу EnrollmentRepository

```

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import ua.opnu.labwork2.enrollment.model.Enrollment;

@Repository
public interface EnrollmentRepository extends JpaRepository<Enrollment, Long> {
    boolean existsByStudentIdAndCourseId(Long studentId, Long courseId);

    boolean existsByStudentIdAndCourseIdAndIdNot(Long studentId, Long courseId,
    Long id);

    long countByCompletionDateIsNull();

    long countByCompletionDateIsNotNull();
}

```

Лістинг В.5 – Код інтерфейсу ModuleRepository

```

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import ua.opnu.labwork2.module.model.CourseModule;

@Repository
public interface ModuleRepository extends JpaRepository<CourseModule, Long> {
    boolean existsByCourseIdAndOrderIndex(Long courseId, Integer orderIndex);

    boolean existsByCourseIdAndOrderIndexAndIdNot(Long courseId, Integer
    orderIndex, Long id);
}

```

ДОДАТОК Г

Вихідний код сервісів

У додатку Г наведено класи сервісного шару. Вони реалізують бізнес-логіку backend-сервісу, перевіряють існування об'єктів, унікальність даних, виконують створення, оновлення, видалення та інші операції.

Лістинг Г.1 – Код класу InstructorService

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import ua.opnu.labwork2.course.model.Course;
import ua.opnu.labwork2.course.repository.CourseRepository;
import ua.opnu.labwork2.exception.DuplicateResourceException;
import ua.opnu.labwork2.exception.ResourceNotFoundException;
import ua.opnu.labwork2.instructor.dto.InstructorCreateRequest;
import ua.opnu.labwork2.instructor.dto.InstructorUpdateRequest;
import ua.opnu.labwork2.instructor.model.Instructor;
import ua.opnu.labwork2.instructor.repository.InstructorRepository;

import java.util.List;

@Service
public class InstructorService {

    @Autowired
    private InstructorRepository repository;

    @Autowired
    private CourseRepository courseRepository;

    public Instructor create(InstructorCreateRequest request) {
        if (repository.existsByEmail(request.email())) {
            throw new DuplicateResourceException("Instructor email already
exists");
        }
        Instructor instructor = new Instructor();
        apply(instructor, request.firstName(), request.lastName(),
request.email(), request.specialization());
        return repository.save(instructor);
    }

    public List<Instructor> getAll() {
        return repository.findAll();
    }

    public Instructor getById(Long id) {
        return repository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Instructor",
id));
    }

    public Instructor update(Long id, InstructorUpdateRequest request) {
        Instructor instructor = getById(id);
```

```

        if (repository.existsByEmailAndIdNot(request.email(), id)) {
            throw new DuplicateResourceException("Instructor email already
exists");
        }
        apply(instructor, request.firstName(), request.lastName(),
request.email(), request.specialization());
        return repository.save(instructor);
    }

    public void delete(Long id) {
        if (!repository.existsById(id)) {
            throw new ResourceNotFoundException("Instructor", id);
        }
        repository.deleteById(id);
    }

    public List<Course> getCourses(Long instructorId) {
        getById(instructorId);
        return courseRepository.findAll()
            .stream()
            .filter(c -> c.getInstructor() != null &&
                c.getInstructor().getId().equals(instructorId))
            .toList();
    }

    private void apply(Instructor instructor, String firstName, String lastName,
String email, String specialization) {
        instructor.setFirstName(firstName);
        instructor.setLastName(lastName);
        instructor.setEmail(email);
        instructor.setSpecialization(specialization);
    }
}

```

ЛІСТИНГ Г.2 – Код класу CourseService

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import ua.opnu.labwork2.course.dto.CourseCreateRequest;
import ua.opnu.labwork2.course.dto.CourseUpdateRequest;
import ua.opnu.labwork2.course.model.Course;
import ua.opnu.labwork2.course.repository.CourseRepository;
import ua.opnu.labwork2.enrollment.model.Enrollment;
import ua.opnu.labwork2.enrollment.repository.EnrollmentRepository;
import ua.opnu.labwork2.exception.ResourceNotFoundException;
import ua.opnu.labwork2.instructor.model.Instructor;
import ua.opnu.labwork2.instructor.repository.InstructorRepository;
import ua.opnu.labwork2.module.model.CourseModule;
import ua.opnu.labwork2.module.repository.ModuleRepository;

import java.util.List;

@Service
public class CourseService {

    @Autowired
    private CourseRepository repository;

    @Autowired

```

```

private InstructorRepository instructorRepository;

@Autowired
private ModuleRepository moduleRepository;

@Autowired
private EnrollmentRepository enrollmentRepository;

public Course create(CourseCreateRequest request) {
    Course course = new Course();
    apply(course, request.title(), request.description(), request.level(),
request.createdDate(), request.instructorId());
    return repository.save(course);
}

public List<Course> getAll() {
    return repository.findAll();
}

public Course getById(Long id) {
    return repository.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException("Course", id));
}

public Course update(Long id, CourseUpdateRequest request) {
    Course course = getById(id);
    apply(course, request.title(), request.description(), request.level(),
request.createdDate(), request.instructorId());
    return repository.save(course);
}

public void delete(Long id) {
    if (!repository.existsById(id)) {
        throw new ResourceNotFoundException("Course", id);
    }
    repository.deleteById(id);
}

public List<Course> getByInstructor(Long instructorId) {
    if (!instructorRepository.existsById(instructorId)) {
        throw new ResourceNotFoundException("Instructor", instructorId);
    }
    return repository.findAll()
        .stream()
        .filter(c -> c.getInstructor() != null &&
            c.getInstructor().getId().equals(instructorId))
        .toList();
}

public List<CourseModule> getModules(Long courseId) {
    getById(courseId);
    return moduleRepository.findAll()
        .stream()
        .filter(m -> m.getCourse() != null &&
            m.getCourse().getId().equals(courseId))
        .toList();
}

public List<Enrollment> getStudents(Long courseId) {
    getById(courseId);
    return enrollmentRepository.findAll()
        .stream()

```

```

        .filter(e -> e.getCourse() != null &&
            e.getCourse().getId().equals(courseId))
        .toList();
    }

    private void apply(Course course, String title, String description, String
level,
        java.time.LocalDate createdAt, Long instructorId) {
        Instructor instructor = instructorRepository.findById(instructorId)
            .orElseThrow(() -> new ResourceNotFoundException("Instructor",
instructorId));
        course.setTitle(title);
        course.setDescription(description);
        course.setLevel(level);
        course.setCreatedAt(createdAt);
        course.setInstructor(instructor);
    }
}

```

ЛІСТИНГ Г.3 – Код класу StudentService

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import ua.opnu.labwork2.enrollment.model.Enrollment;
import ua.opnu.labwork2.enrollment.repository.EnrollmentRepository;
import ua.opnu.labwork2.exception.DuplicateResourceException;
import ua.opnu.labwork2.exception.ResourceNotFoundException;
import ua.opnu.labwork2.student.dto.StudentCreateRequest;
import ua.opnu.labwork2.student.dto.StudentUpdateRequest;
import ua.opnu.labwork2.student.model.Student;
import ua.opnu.labwork2.student.repository.StudentRepository;

import java.util.List;

@Service
public class StudentService {

    @Autowired
    private StudentRepository repository;

    @Autowired
    private EnrollmentRepository enrollmentRepository;

    public Student create(StudentCreateRequest request) {
        if (repository.existsByEmail(request.email())) {
            throw new DuplicateResourceException("Student email already exists");
        }
        Student student = new Student();
        apply(student, request);
        return repository.save(student);
    }

    public List<Student> getAll() {
        return repository.findAll();
    }

    public Student getById(Long id) {
        return repository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Student", id));
    }
}

```



```

    }

    public Student update(Long id, StudentUpdateRequest request) {
        Student student = getById(id);
        if (repository.existsByEmailAndIdNot(request.email(), id)) {
            throw new DuplicateResourceException("Student email already exists");
        }
        student.setFirstName(request.firstName());
        student.setLastName(request.lastName());
        student.setEmail(request.email());
        student.setRegistrationDate(request.registrationDate());
        return repository.save(student);
    }

    public void delete(Long id) {
        if (!repository.existsById(id)) {
            throw new ResourceNotFoundException("Student", id);
        }
        repository.deleteById(id);
    }

    public List<Enrollment> getCourses(Long studentId) {
        getById(studentId);
        return enrollmentRepository.findAll()
            .stream()
            .filter(e -> e.getStudent() != null &&
                e.getStudent().getId().equals(studentId))
            .toList();
    }

    private void apply(Student student, StudentCreateRequest request) {
        student.setFirstName(request.firstName());
        student.setLastName(request.lastName());
        student.setEmail(request.email());
        student.setRegistrationDate(request.registrationDate());
    }
}

```

ЛІСТИНГ Г.4 – Код класу EnrollmentService

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import ua.opnu.labwork2.course.model.Course;
import ua.opnu.labwork2.course.repository.CourseRepository;
import ua.opnu.labwork2.enrollment.dto.EnrollmentCreateRequest;
import ua.opnu.labwork2.enrollment.dto.EnrollmentProgressUpdateRequest;
import ua.opnu.labwork2.enrollment.dto.EnrollmentUpdateRequest;
import ua.opnu.labwork2.enrollment.model.Enrollment;
import ua.opnu.labwork2.enrollment.repository.EnrollmentRepository;
import ua.opnu.labwork2.exception.BadRequestException;
import ua.opnu.labwork2.exception.CourseEnrollmentException;
import ua.opnu.labwork2.exception.ResourceNotFoundException;
import ua.opnu.labwork2.student.model.Student;
import ua.opnu.labwork2.student.repository.StudentRepository;

import java.time.LocalDate;
import java.util.List;

@Service

```

```

public class EnrollmentService {

    @Autowired
    private EnrollmentRepository repository;

    @Autowired
    private StudentRepository studentRepository;

    @Autowired
    private CourseRepository courseRepository;

    public Enrollment create(EnrollmentCreateRequest request) {
        validateCompletionDate(request.enrollmentDate(),
request.completionDate());
        if (repository.existsByStudentIdAndCourseId(request.studentId(),
request.courseId())) {
            throw new CourseEnrollmentException("Student is already enrolled in
this course");
        }

        Student student = studentRepository.findById(request.studentId())
            .orElseThrow(() -> new ResourceNotFoundException("Student",
request.studentId()));
        Course course = courseRepository.findById(request.courseId())
            .orElseThrow(() -> new ResourceNotFoundException("Course",
request.courseId()));

        Enrollment enrollment = new Enrollment();
        enrollment.setStudent(student);
        enrollment.setCourse(course);
        enrollment.setEnrollmentDate(request.enrollmentDate());
        enrollment.setCompletionDate(request.completionDate());
        enrollment.setProgress(request.progress());
        return repository.save(enrollment);
    }

    public List<Enrollment> getAll() {
        return repository.findAll();
    }

    public Enrollment getById(Long id) {
        return repository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Enrollment",
id));
    }

    public Enrollment update(Long id, EnrollmentUpdateRequest request) {
        validateCompletionDate(request.enrollmentDate(),
request.completionDate());
        Enrollment enrollment = getById(id);
        if (repository.existsByStudentIdAndCourseIdAndIdNot(request.studentId(),
request.courseId(), id)) {
            throw new CourseEnrollmentException("Student is already enrolled in
this course");
        }

        Student student = studentRepository.findById(request.studentId())
            .orElseThrow(() -> new ResourceNotFoundException("Student",
request.studentId()));
        Course course = courseRepository.findById(request.courseId())
            .orElseThrow(() -> new ResourceNotFoundException("Course",
request.courseId()));
    }
}

```

```

        enrollment.setStudent(student);
        enrollment.setCourse(course);
        enrollment.setEnrollmentDate(request.enrollmentDate());
        enrollment.setCompletionDate(request.completionDate());
        enrollment.setProgress(request.progress());
        return repository.save(enrollment);
    }

    public void delete(Long id) {
        if (!repository.existsById(id)) {
            throw new ResourceNotFoundException("Enrollment", id);
        }
        repository.deleteById(id);
    }

    public Enrollment updateProgress(Long id, EnrollmentProgressUpdateRequest
request) {
        Enrollment enrollment = getById(id);
        enrollment.setProgress(request.progress());
        return repository.save(enrollment);
    }

    private void validateCompletionDate(LocalDate enrollmentDate, LocalDate
completionDate) {
        if (completionDate != null && enrollmentDate != null &&
completionDate.isBefore(enrollmentDate)) {
            throw new BadRequestException("Completion date cannot be before
enrollment date");
        }
    }
}

```

Лістинг Г.5 – Код класу ModuleService

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import ua.opnu.labwork2.course.model.Course;
import ua.opnu.labwork2.course.repository.CourseRepository;
import ua.opnu.labwork2.exception.ModuleOrderException;
import ua.opnu.labwork2.exception.ResourceNotFoundException;
import ua.opnu.labwork2.module.dto.ModuleCreateRequest;
import ua.opnu.labwork2.module.dto.ModuleUpdateRequest;
import ua.opnu.labwork2.module.model.CourseModule;
import ua.opnu.labwork2.module.repository.ModuleRepository;

import java.util.List;

@Service
public class ModuleService {

    @Autowired
    private ModuleRepository repository;

    @Autowired
    private CourseRepository courseRepository;

    public CourseModule create(ModuleCreateRequest request) {
        if (repository.existsByCourseIdAndOrderIndex(request.courseId(),

```

```

request.orderIndex())) {
    throw new ModuleOrderException("Module with this order index already
exists in the course");
}
CourseModule module = new CourseModule();
apply(module, request.title(), request.description(),
request.orderIndex(), request.courseId());
return repository.save(module);
}

public List<CourseModule> getAll() {
    return repository.findAll();
}

public CourseModule getById(Long id) {
    return repository.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException("Module", id));
}

public CourseModule update(Long id, ModuleUpdateRequest request) {
    CourseModule module = getById(id);
    if (repository.existsByCourseIdAndOrderIndexAndIdNot(request.courseId(),
request.orderIndex(), id)) {
        throw new ModuleOrderException("Module with this order index already
exists in the course");
    }
    apply(module, request.title(), request.description(),
request.orderIndex(), request.courseId());
    return repository.save(module);
}

public void delete(Long id) {
    if (!repository.existsById(id)) {
        throw new ResourceNotFoundException("Module", id);
    }
    repository.deleteById(id);
}

public CourseModule addToCourse(Long courseId, Long moduleId) {
    CourseModule module = getById(moduleId);
    Course course = courseRepository.findById(courseId)
        .orElseThrow(() -> new ResourceNotFoundException("Course",
courseId));
    if (repository.existsByCourseIdAndOrderIndexAndIdNot(courseId,
module.getOrderIndex(), moduleId)) {
        throw new ModuleOrderException("Module with this order index already
exists in the course");
    }
    module.setCourse(course);
    return repository.save(module);
}

public void removeFromCourse(Long courseId, Long moduleId) {
    if (!courseRepository.existsById(courseId)) {
        throw new ResourceNotFoundException("Course", courseId);
    }
    CourseModule module = getById(moduleId);
    if (module.getCourse() == null ||
!module.getCourse().getId().equals(courseId)) {
        throw new ResourceNotFoundException("Module in course", moduleId);
    }
    module.setCourse(null);
}

```

```
        repository.save(module);
    }

    private void apply(CourseModule module, String title, String description,
Integer orderIndex, Long courseId) {
        Course course = courseRepository.findById(courseId)
            .orElseThrow(() -> new ResourceNotFoundException("Course",
courseId));
        module.setTitle(title);
        module.setDescription(description);
        module.setOrderIndex(orderIndex);
        module.setCourse(course);
    }
}
```

ДОДАТОК Д

Вихідний код контролерів

У додатку Д наведено REST-контролери backend-сервісу. Вони приймають HTTP-запити, викликають відповідні методи сервісів або репозиторіїв та повертають відповіді клієнту.

Лістинг Д.1 – Код класу InstructorController

```
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.media.ArraySchema;
import io.swagger.v3.oas.annotations.media.Content;
import io.swagger.v3.oas.annotations.media.Schema;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import ua.opnu.labwork2.course.dto.CourseResponse;
import ua.opnu.labwork2.course.model.Course;
import ua.opnu.labwork2.exception.ApiErrorResponse;
import ua.opnu.labwork2.instructor.dto.InstructorCreateRequest;
import ua.opnu.labwork2.instructor.dto.InstructorResponse;
import ua.opnu.labwork2.instructor.dto.InstructorUpdateRequest;
import ua.opnu.labwork2.instructor.model.Instructor;
import ua.opnu.labwork2.instructor.service.InstructorService;

import java.util.List;

@RestController
@RequestMapping("/instructors")
@Tag(name = "Викладачі", description = "Управління викладачами та курсами, які вони проводять")
public class InstructorController {

    @Autowired
    private InstructorService service;

    @Operation(summary = "Створити викладача", description = "Створює профіль викладача. Перевіряються ім'я, прізвище, унікальний email та спеціалізація.")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Викладача створено",
            content = @Content(schema = @Schema(implementation = InstructorResponse.class))),
        @ApiResponse(responseCode = "400", description = "Некоректні дані викладача або помилки валідації", content = @Content(schema =
```

```

@Schema(implementation = ApiErrorResponse.class))),
    @ApiResponse(responseCode = "409", description = "Викладач з таким
email вже існує", content = @Content(schema = @Schema(implementation =
ApiErrorResponse.class))),
    @ApiResponse(responseCode = "500", description = "Внутрішня помилка
сервера", content = @Content(schema = @Schema(implementation =
ApiErrorResponse.class)))
    })
    @PostMapping
    public ResponseEntity<InstructorResponse> create(@Valid @RequestBody
InstructorCreateRequest request) {
        return ResponseEntity.ok(toResponse(service.create(request)));
    }

    @Operation(summary = "Отримати всіх викладачів", description = "Повертає
список усіх викладачів системи.")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Список викладачів
отримано", content = @Content(array = @ArraySchema(schema = @Schema(implementation
= InstructorResponse.class)))),
        @ApiResponse(responseCode = "500", description = "Внутрішня помилка
сервера", content = @Content(schema = @Schema(implementation =
ApiErrorResponse.class)))
    })
    @GetMapping
    public ResponseEntity<List<InstructorResponse>> getAll() {
        return
ResponseEntity.ok(service.getAll().stream().map(this::toResponse).toList());
    }

    @Operation(summary = "Отримати викладача за id", description = "Повертає
профіль викладача за ідентифікатором.")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Викладача знайдено",
content = @Content(schema = @Schema(implementation = InstructorResponse.class))),
        @ApiResponse(responseCode = "404", description = "Викладача не
знайдено", content = @Content(schema = @Schema(implementation =
ApiErrorResponse.class))),
        @ApiResponse(responseCode = "500", description = "Внутрішня помилка
сервера", content = @Content(schema = @Schema(implementation =
ApiErrorResponse.class)))
    })
    @GetMapping("/{id}")
    public ResponseEntity<InstructorResponse> getById(@PathVariable Long id) {
        return ResponseEntity.ok(toResponse(service.getById(id)));
    }

    @Operation(summary = "Оновити викладача", description = "Оновлює профіль
викладача. Перевіряються обов'язкові поля, формат email та унікальність email.")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Викладача оновлено",
content = @Content(schema = @Schema(implementation = InstructorResponse.class))),
        @ApiResponse(responseCode = "400", description = "Некоректні дані
викладача або помилки валідації", content = @Content(schema =
@Schema(implementation = ApiErrorResponse.class))),
        @ApiResponse(responseCode = "404", description = "Викладача не
знайдено", content = @Content(schema = @Schema(implementation =
ApiErrorResponse.class))),
        @ApiResponse(responseCode = "409", description = "Викладач з таким
email вже існує", content = @Content(schema = @Schema(implementation =
ApiErrorResponse.class))),
        @ApiResponse(responseCode = "500", description = "Внутрішня помилка

```

```

сервера", content = @Content(schema = @Schema(implementation =
ApiResponse.class)))
    })
    @PutMapping("/{id}")
    public ResponseEntity<InstructorResponse> update(
        @PathVariable Long id,
        @Valid @RequestBody InstructorUpdateRequest request) {
        return ResponseEntity.ok(toResponse(service.update(id, request)));
    }

    @Operation(summary = "Видалити викладача", description = "Видаляє викладача за
ідентифікатором. Якщо викладач має курси, може бути повернутий конфлікт.")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Викладача видалено",
content = @Content(schema = @Schema(implementation = String.class))),
        @ApiResponse(responseCode = "404", description = "Викладача не
знайдено", content = @Content(schema = @Schema(implementation =
ApiResponse.class))),
        @ApiResponse(responseCode = "409", description = "Викладача не можна
видалити через пов'язані курси", content = @Content(schema =
@Schema(implementation = ApiResponse.class))),
        @ApiResponse(responseCode = "500", description = "Внутрішня помилка
сервера", content = @Content(schema = @Schema(implementation =
ApiResponse.class)))
    })
    @DeleteMapping("/{id}")
    public ResponseEntity<String> delete(@PathVariable Long id) {
        service.delete(id);
        return ResponseEntity.ok("Deleted instructor " + id);
    }

    @Operation(summary = "Отримати курси викладача", description = "Повертає
курси, які закріплені за конкретним викладачем.")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Курси викладача
отримано", content = @Content(array = @ArraySchema(schema = @Schema(implementation
= CourseResponse.class))),
        @ApiResponse(responseCode = "404", description = "Викладача не
знайдено", content = @Content(schema = @Schema(implementation =
ApiResponse.class))),
        @ApiResponse(responseCode = "500", description = "Внутрішня помилка
сервера", content = @Content(schema = @Schema(implementation =
ApiResponse.class)))
    })
    @GetMapping("/{id}/courses")
    public ResponseEntity<List<CourseResponse>> getCourses(@PathVariable Long id)
{
    return
ResponseEntity.ok(service.getCourses(id).stream().map(this::toCourseResponse).toLi
st());
}

    private InstructorResponse toResponse(Instructor instructor) {
        return new InstructorResponse(
            instructor.getId(),
            instructor.getFirstName(),
            instructor.getLastName(),
            instructor.getEmail(),
            instructor.getSpecialization()
        );
    }
}

```



```

    private CourseResponse toCourseResponse(Course course) {
        return new CourseResponse(
            course.getId(),
            course.getTitle(),
            course.getDescription(),
            course.getLevel(),
            course.getCreatedDate(),
            course.getInstructor() == null ? null :
course.getInstructor().getId()
        );
    }
}

```

Лістинг Д.2 – Код класу CourseController

```

import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.media.ArraySchema;
import io.swagger.v3.oas.annotations.media.Content;
import io.swagger.v3.oas.annotations.media.Schema;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import ua.opnu.labwork2.course.dto.CourseCreateRequest;
import ua.opnu.labwork2.course.dto.CourseResponse;
import ua.opnu.labwork2.course.dto.CourseUpdateRequest;
import ua.opnu.labwork2.course.model.Course;
import ua.opnu.labwork2.course.servise.CourseService;
import ua.opnu.labwork2.enrollment.model.Enrollment;
import ua.opnu.labwork2.exception.ApiErrorResponse;
import ua.opnu.labwork2.module.dto.ModuleResponse;
import ua.opnu.labwork2.module.model.CourseModule;
import ua.opnu.labwork2.student.dto.StudentResponse;
import ua.opnu.labwork2.student.model.Student;

import java.util.List;

@RestController
@RequestMapping("/courses")
@Tag(name = "Курси", description = "Управління онлайн-курсами, їх викладачами, модулями та студентами")
public class CourseController {

    @Autowired
    private CourseService service;

    @Operation(summary = "Створити курс", description = "Створює новий курс. Перевіряється назва, опис, рівень складності, дата створення та існування викладача.")

```

```

    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Курс успішно
створено", content = @Content(schema = @Schema(implementation =
CourseResponse.class))),
        @ApiResponse(responseCode = "400", description = "Некоректні дані
курсу або помилки валідації", content = @Content(schema = @Schema(implementation =
ApiErrorResponse.class))),
        @ApiResponse(responseCode = "404", description = "Викладача для курсу
не знайдено", content = @Content(schema = @Schema(implementation =
ApiErrorResponse.class))),
        @ApiResponse(responseCode = "409", description = "Конфлікт бізнес-
правил під час створення курсу", content = @Content(schema =
@Schema(implementation = ApiErrorResponse.class))),
        @ApiResponse(responseCode = "500", description = "Внутрішня помилка
сервера", content = @Content(schema = @Schema(implementation =
ApiErrorResponse.class)))
    })
    @PostMapping
    public ResponseEntity<CourseResponse> create(@Valid @RequestBody
CourseCreateRequest request) {
        return ResponseEntity.ok(toResponse(service.create(request)));
    }

    @Operation(summary = "Отримати всі курси", description = "Повертає список усіх
курсів, зареєстрованих у системі.")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Список курсів
отримано", content = @Content(array = @ArraySchema(schema = @Schema(implementation
= CourseResponse.class)))),
        @ApiResponse(responseCode = "500", description = "Внутрішня помилка
сервера", content = @Content(schema = @Schema(implementation =
ApiErrorResponse.class)))
    })
    @GetMapping
    public ResponseEntity<List<CourseResponse>> getAll() {
        return
ResponseEntity.ok(service.getAll().stream().map(this::toResponse).toList());
    }

    @Operation(summary = "Отримати курс за id", description = "Повертає один курс
за ідентифікатором. Якщо курс не існує, повертається помилка 404.")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Курс знайдено",
content = @Content(schema = @Schema(implementation = CourseResponse.class))),
        @ApiResponse(responseCode = "404", description = "Курс не знайдено",
content = @Content(schema = @Schema(implementation = ApiErrorResponse.class))),
        @ApiResponse(responseCode = "500", description = "Внутрішня помилка
сервера", content = @Content(schema = @Schema(implementation =
ApiErrorResponse.class)))
    })
    @GetMapping("/{id}")
    public ResponseEntity<CourseResponse> getById(@PathVariable Long id) {
        return ResponseEntity.ok(toResponse(service.getById(id)));
    }

    @Operation(summary = "Оновити курс", description = "Оновлює дані курсу.
Перевіряється валідність DTO, існування курсу та викладача.")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Курс оновлено",
content = @Content(schema = @Schema(implementation = CourseResponse.class))),
        @ApiResponse(responseCode = "400", description = "Некоректні дані
курсу або помилки валідації", content = @Content(schema = @Schema(implementation =

```

```

ApiResponse.class))),
    @ApiResponse(responseCode = "404", description = "Курс або викладача
не знайдено", content = @Content(schema = @Schema(implementation =
ApiResponse.class))),
    @ApiResponse(responseCode = "409", description = "Конфлікт бізнес-
правил під час оновлення курсу", content = @Content(schema =
@Schema(implementation = ApiResponse.class))),
    @ApiResponse(responseCode = "500", description = "Внутрішня помилка
сервера", content = @Content(schema = @Schema(implementation =
ApiResponse.class)))
})
@PutMapping("/{id}")
public ResponseEntity<CourseResponse> update(
    @PathVariable Long id,
    @Valid @RequestBody CourseUpdateRequest request) {
    return ResponseEntity.ok(toResponse(service.update(id, request)));
}

@Operation(summary = "Видалити курс", description = "Видаляє курс за
ідентифікатором. Якщо курс має пов'язані дані, сервіс може повернути конфлікт
бізнес-правил.")
@ApiResponses({
    @ApiResponse(responseCode = "200", description = "Курс видалено",
content = @Content(schema = @Schema(implementation = String.class))),
    @ApiResponse(responseCode = "404", description = "Курс не знайдено",
content = @Content(schema = @Schema(implementation = ApiResponse.class))),
    @ApiResponse(responseCode = "409", description = "Курс не можна
видалити через пов'язані записи", content = @Content(schema =
@Schema(implementation = ApiResponse.class))),
    @ApiResponse(responseCode = "500", description = "Внутрішня помилка
сервера", content = @Content(schema = @Schema(implementation =
ApiResponse.class)))
})
@DeleteMapping("/{id}")
public ResponseEntity<String> delete(@PathVariable Long id) {
    service.delete(id);
    return ResponseEntity.ok("Deleted course " + id);
}

@Operation(summary = "Отримати курси викладача", description = "Повертає всі
курси, які веде конкретний викладач.")
@ApiResponses({
    @ApiResponse(responseCode = "200", description = "Список курсів
викладача отримано", content = @Content(array = @ArraySchema(schema =
@Schema(implementation = CourseResponse.class)))),
    @ApiResponse(responseCode = "404", description = "Викладача не
знайдено", content = @Content(schema = @Schema(implementation =
ApiResponse.class))),
    @ApiResponse(responseCode = "500", description = "Внутрішня помилка
сервера", content = @Content(schema = @Schema(implementation =
ApiResponse.class)))
})
@GetMapping("/instructor/{instructorId}")
public ResponseEntity<List<CourseResponse>> getByInstructor(@PathVariable Long
instructorId) {
    return
ResponseEntity.ok(service.getByInstructor(instructorId).stream().map(this::toRespo
nse).toList());
}

@Operation(summary = "Отримати модулі курсу", description = "Повертає
навчальні модулі конкретного курсу.")

```

```

    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Модулі курсу  
отримано", content = @Content(array = @ArraySchema(schema = @Schema(implementation  
= ModuleResponse.class)))),
        @ApiResponse(responseCode = "404", description = "Курс не знайдено",  
content = @Content(schema = @Schema(implementation = ApiErrorResponse.class))),
        @ApiResponse(responseCode = "500", description = "Внутрішня помилка  
сервера", content = @Content(schema = @Schema(implementation =  
ApiErrorResponse.class)))
    })
    @GetMapping("/{id}/modules")
    public ResponseEntity<List<ModuleResponse>> getModules(@PathVariable Long id)
    {
        return
ResponseEntity.ok(service.getModules(id).stream().map(this::toModuleResponse).toLi  
st());
    }

    @Operation(summary = "Отримати студентів курсу", description = "Повертає  
студентів, які зараховані на конкретний курс.")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Студентів курсу  
отримано", content = @Content(array = @ArraySchema(schema = @Schema(implementation  
= StudentResponse.class)))),
        @ApiResponse(responseCode = "404", description = "Курс не знайдено",  
content = @Content(schema = @Schema(implementation = ApiErrorResponse.class))),
        @ApiResponse(responseCode = "500", description = "Внутрішня помилка  
сервера", content = @Content(schema = @Schema(implementation =  
ApiErrorResponse.class)))
    })
    @GetMapping("/{id}/students")
    public ResponseEntity<List<StudentResponse>> getStudents(@PathVariable Long  
id) {
        return ResponseEntity.ok(service.getStudents(id).stream()
            .map(Enrollment::getStudent)
            .map(this::toStudentResponse)
            .toList());
    }

    private CourseResponse toResponse(Course course) {
        return new CourseResponse(
            course.getId(),
            course.getTitle(),
            course.getDescription(),
            course.getLevel(),
            course.getCreatedDate(),
            course.getInstructor() == null ? null :  
course.getInstructor().getId()
        );
    }

    private ModuleResponse toModuleResponse(CourseModule module) {
        return new ModuleResponse(
            module.getId(),
            module.getTitle(),
            module.getDescription(),
            module.getOrderIndex(),
            module.getCourse() == null ? null : module.getCourse().getId()
        );
    }

    private StudentResponse toStudentResponse(Student student) {

```

```

        return new StudentResponse(
            student.getId(),
            student.getFirstName(),
            student.getLastName(),
            student.getEmail(),
            student.getRegistrationDate()
        );
    }
}

```

Лістинг Д.3 – Код класу StudentController

```

import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.media.ArraySchema;
import io.swagger.v3.oas.annotations.media.Content;
import io.swagger.v3.oas.annotations.media.Schema;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import ua.opnu.labwork2.course.dto.CourseResponse;
import ua.opnu.labwork2.course.model.Course;
import ua.opnu.labwork2.enrollment.model.Enrollment;
import ua.opnu.labwork2.exception.ApiErrorResponse;
import ua.opnu.labwork2.student.dto.StudentCreateRequest;
import ua.opnu.labwork2.student.dto.StudentResponse;
import ua.opnu.labwork2.student.dto.StudentUpdateRequest;
import ua.opnu.labwork2.student.model.Student;
import ua.opnu.labwork2.student.service.StudentService;

import java.util.List;

@RestController
@RequestMapping("/students")
@Tag(name = "Студенти", description = "Управління студентами та курсами, на які вони записані")
public class StudentController {
    @Autowired
    private StudentService service;

    @Operation(summary = "Створити студента", description = "Створює профіль студента. Перевіряються ім'я, прізвище, унікальний email та дата реєстрації.")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Студента створено",
            content = @Content(schema = @Schema(implementation = StudentResponse.class))),
        @ApiResponse(responseCode = "400", description = "Некоректні дані студента або помилки валідації", content = @Content(schema =
            @Schema(implementation = ApiErrorResponse.class))),
        @ApiResponse(responseCode = "409", description = "Студент з таким email вже існує", content = @Content(schema = @Schema(implementation =

```

```

ApiResponse.class))),
        @ApiResponse(responseCode = "500", description = "Внутрішня помилка
сервера", content = @Content(schema = @Schema(implementation =
ApiResponse.class))))
    })
    @PostMapping
    public ResponseEntity<StudentResponse> create(@Valid @RequestBody
StudentCreateRequest request) {
        return ResponseEntity.ok(toResponse(service.create(request)));
    }

    @Operation(summary = "Отримати всіх студентів", description = "Повертає список
усіх студентів, зареєстрованих у системі.")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Список студентів
отримано", content = @Content(array = @ArraySchema(schema = @Schema(implementation
= StudentResponse.class)))),
        @ApiResponse(responseCode = "500", description = "Внутрішня помилка
сервера", content = @Content(schema = @Schema(implementation =
ApiResponse.class))))
    })
    @GetMapping
    public ResponseEntity<List<StudentResponse>> getAll() {
        return
ResponseEntity.ok(service.getAll().stream().map(this::toResponse).toList());
    }

    @Operation(summary = "Отримати студента за id", description = "Повертає
профіль студента за ідентифікатором.")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Студента знайдено",
content = @Content(schema = @Schema(implementation = StudentResponse.class))),
        @ApiResponse(responseCode = "404", description = "Студента не
знайдено", content = @Content(schema = @Schema(implementation =
ApiResponse.class))),
        @ApiResponse(responseCode = "500", description = "Внутрішня помилка
сервера", content = @Content(schema = @Schema(implementation =
ApiResponse.class))))
    })
    @GetMapping("/{id}")
    public ResponseEntity<StudentResponse> getById(@PathVariable Long id) {
        return ResponseEntity.ok(toResponse(service.getById(id)));
    }

    @Operation(summary = "Оновити студента", description = "Оновлює профіль
студента. Перевіряються обов'язкові поля, формат email та унікальність email.")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Студента оновлено",
content = @Content(schema = @Schema(implementation = StudentResponse.class))),
        @ApiResponse(responseCode = "400", description = "Некоректні дані
студента або помилки валідації", content = @Content(schema =
@Schema(implementation = ApiResponse.class))),
        @ApiResponse(responseCode = "404", description = "Студента не
знайдено", content = @Content(schema = @Schema(implementation =
ApiResponse.class))),
        @ApiResponse(responseCode = "409", description = "Студент з таким
email вже існує", content = @Content(schema = @Schema(implementation =
ApiResponse.class))),
        @ApiResponse(responseCode = "500", description = "Внутрішня помилка
сервера", content = @Content(schema = @Schema(implementation =
ApiResponse.class))))
    })

```

```

@PutMapping("/{id}")
public ResponseEntity<StudentResponse> update(
    @PathVariable Long id,
    @Valid @RequestBody StudentUpdateRequest request) {
    return ResponseEntity.ok(toResponse(service.update(id, request)));
}

@Operation(summary = "Видалити студента", description = "Видаляє студента за ідентифікатором. За наявності пов'язаних записів сервіс може повернути конфлікт.")
@ApiResponses({
    @ApiResponse(responseCode = "200", description = "Студента видалено", content = @Content(schema = @Schema(implementation = String.class))),
    @ApiResponse(responseCode = "404", description = "Студента не знайдено", content = @Content(schema = @Schema(implementation = ApiErrorResponse.class))),
    @ApiResponse(responseCode = "409", description = "Студента не можна видалити через пов'язані записи", content = @Content(schema = @Schema(implementation = ApiErrorResponse.class))),
    @ApiResponse(responseCode = "500", description = "Внутрішня помилка сервера", content = @Content(schema = @Schema(implementation = ApiErrorResponse.class)))
})
@DeleteMapping("/{id}")
public ResponseEntity<String> delete(@PathVariable Long id) {
    service.delete(id);
    return ResponseEntity.ok("Deleted student " + id);
}

@Operation(summary = "Отримати курси студента", description = "Повертає всі курси, на які записаний студент.")
@ApiResponses({
    @ApiResponse(responseCode = "200", description = "Курси студента отримано", content = @Content(array = @ArraySchema(schema = @Schema(implementation = CourseResponse.class)))),
    @ApiResponse(responseCode = "404", description = "Студента не знайдено", content = @Content(schema = @Schema(implementation = ApiErrorResponse.class))),
    @ApiResponse(responseCode = "500", description = "Внутрішня помилка сервера", content = @Content(schema = @Schema(implementation = ApiErrorResponse.class)))
})
@GetMapping("/{id}/courses")
public ResponseEntity<List<CourseResponse>> getCourses(@PathVariable Long id) {
    return ResponseEntity.ok(service.getCourses(id).stream()
        .map(Enrollment::getCourse)
        .map(this::toCourseResponse)
        .toList());
}

private StudentResponse toResponse(Student student) {
    return new StudentResponse(
        student.getId(),
        student.getFirstName(),
        student.getLastName(),
        student.getEmail(),
        student.getRegistrationDate()
    );
}

private CourseResponse toCourseResponse(Course course) {
    return new CourseResponse(

```

```

        course.getId(),
        course.getTitle(),
        course.getDescription(),
        course.getLevel(),
        course.getCreatedDate(),
        course.getInstructor() == null ? null :
course.getInstructor().getId()
    );
}
}

```

Лістинг Д.4 – Код класу EnrollmentController

```

import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.media.ArraySchema;
import io.swagger.v3.oas.annotations.media.Content;
import io.swagger.v3.oas.annotations.media.Schema;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import ua.opnu.labwork2.enrollment.dto.EnrollmentCreateRequest;
import ua.opnu.labwork2.enrollment.dto.EnrollmentProgressUpdateRequest;
import ua.opnu.labwork2.enrollment.dto.EnrollmentResponse;
import ua.opnu.labwork2.enrollment.model.Enrollment;
import ua.opnu.labwork2.enrollment.service.EnrollmentService;
import ua.opnu.labwork2.exception.ApiErrorResponse;

import java.util.List;

@RestController
@RequestMapping("/enrollments")
@Tag(name = "Записи на курси", description = "Управління записами студентів на онлайн-курси та прогресом навчання")
public class EnrollmentController {

    @Autowired
    private EnrollmentService service;

    @Operation(summary = "Записати студента на курс", description = "Створює запис студента на курс. Перевіряється існування студента і курсу, унікальність запису та коректність дат.")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Запис створено",
            content = @Content(schema = @Schema(implementation = EnrollmentResponse.class))),
        @ApiResponse(responseCode = "400", description = "Некоректні дані запису або помилки валідації", content = @Content(schema = @Schema(implementation = ApiErrorResponse.class))),
        @ApiResponse(responseCode = "404", description = "Студента або курс не знайдено", content = @Content(schema = @Schema(implementation = ApiErrorResponse.class))),
        @ApiResponse(responseCode = "409", description = "Студент вже записаний на цей курс", content = @Content(schema = @Schema(implementation = ApiErrorResponse.class))),
        @ApiResponse(responseCode = "500", description = "Внутрішня помилка сервера", content = @Content(schema = @Schema(implementation =

```



```

ApiResponse.class)))
    })
    @PostMapping
    public ResponseEntity<EnrollmentResponse> create(@Valid @RequestBody
EnrollmentCreateRequest request) {
        return ResponseEntity.ok(toResponse(service.create(request)));
    }

    @Operation(summary = "Отримати всі записи", description = "Повертає список
усіх записів студентів на курси.")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Список записів
отримано", content = @Content(array = @ArraySchema(schema = @Schema(implementation
= EnrollmentResponse.class)))),
        @ApiResponse(responseCode = "500", description = "Внутрішня помилка
сервера", content = @Content(schema = @Schema(implementation =
ApiResponse.class)))
    })
    @GetMapping
    public ResponseEntity<List<EnrollmentResponse>> getAll() {
        return
ResponseEntity.ok(service.getAll().stream().map(this::toResponse).toList());
    }

    @Operation(summary = "Отримати запис за id", description = "Повертає запис
студента на курс за ідентифікатором.")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Запис знайдено",
content = @Content(schema = @Schema(implementation = EnrollmentResponse.class))),
        @ApiResponse(responseCode = "404", description = "Запис не знайдено",
content = @Content(schema = @Schema(implementation = ApiResponse.class))),
        @ApiResponse(responseCode = "500", description = "Внутрішня помилка
сервера", content = @Content(schema = @Schema(implementation =
ApiResponse.class)))
    })
    @GetMapping("/{id}")
    public ResponseEntity<EnrollmentResponse> getById(@PathVariable Long id) {
        return ResponseEntity.ok(toResponse(service.getById(id)));
    }

    @Operation(summary = "Оновити прогрес", description = "Оновлює відсоток
проходження курсу для конкретного запису. Значення прогресу має бути від 0 до
100.")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Прогрес оновлено",
content = @Content(schema = @Schema(implementation = EnrollmentResponse.class))),
        @ApiResponse(responseCode = "400", description = "Некоректне значення
прогресу або помилки валідації", content = @Content(schema =
@Schema(implementation = ApiResponse.class))),
        @ApiResponse(responseCode = "404", description = "Запис не знайдено",
content = @Content(schema = @Schema(implementation = ApiResponse.class))),
        @ApiResponse(responseCode = "500", description = "Внутрішня помилка
сервера", content = @Content(schema = @Schema(implementation =
ApiResponse.class)))
    })
    @PutMapping("/{id}/progress")
    public ResponseEntity<EnrollmentResponse> updateProgress(
        @PathVariable Long id,
        @Valid @RequestBody EnrollmentProgressUpdateRequest request) {
        return ResponseEntity.ok(toResponse(service.updateProgress(id, request)));
    }
}

```

```

        @Operation(summary = "Скасувати запис", description = "Видаляє запис студента
на курс за ідентифікатором.")
        @ApiResponses({
            @ApiResponse(responseCode = "200", description = "Запис видалено",
content = @Content(schema = @Schema(implementation = String.class))),
            @ApiResponse(responseCode = "404", description = "Запис не знайдено",
content = @Content(schema = @Schema(implementation = ApiErrorResponse.class))),
            @ApiResponse(responseCode = "500", description = "Внутрішня помилка
сервера", content = @Content(schema = @Schema(implementation =
ApiErrorResponse.class)))
        })
        @DeleteMapping("/{id}")
        public ResponseEntity<String> delete(@PathVariable Long id) {
            service.delete(id);
            return ResponseEntity.ok("Deleted enrollment " + id);
        }

        private EnrollmentResponse toResponse(Enrollment enrollment) {
            return new EnrollmentResponse(
                enrollment.getId(),
                enrollment.getStudent() == null ? null :
enrollment.getStudent().getId(),
                enrollment.getCourse() == null ? null :
enrollment.getCourse().getId(),
                enrollment.getEnrollmentDate(),
                enrollment.getCompletionDate(),
                enrollment.getProgress()
            );
        }
    }
}

```

Лістинг Д.5 – Код класу ModuleController

```

import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.media.ArraySchema;
import io.swagger.v3.oas.annotations.media.Content;
import io.swagger.v3.oas.annotations.media.Schema;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import ua.opnu.labwork2.exception.ApiErrorResponse;
import ua.opnu.labwork2.module.dto.ModuleCreateRequest;
import ua.opnu.labwork2.module.dto.ModuleResponse;
import ua.opnu.labwork2.module.dto.ModuleUpdateRequest;
import ua.opnu.labwork2.module.model.CourseModule;
import ua.opnu.labwork2.module.servise.ModuleService;

import java.util.List;

@RestController
@Tag(name = "Модулі", description = "Управління навчальними модулями онлайн-
курсів")
public class ModuleController {

```

```

@Autowired
private ModuleService service;

@Operation(summary = "Створити модуль", description = "Створює навчальний
модуль у межах курсу. Перевіряється існування курсу та унікальність порядкового
номера модуля в курсі.")
@ApiResponses({
    @ApiResponse(responseCode = "200", description = "Модуль створено",
content = @Content(schema = @Schema(implementation = ModuleResponse.class))),
    @ApiResponse(responseCode = "400", description = "Некоректні дані
модуля або помилки валідації", content = @Content(schema = @Schema(implementation
= ApiErrorResponse.class))),
    @ApiResponse(responseCode = "404", description = "Курс не знайдено",
content = @Content(schema = @Schema(implementation = ApiErrorResponse.class))),
    @ApiResponse(responseCode = "409", description = "Модуль з таким
порядковим номером вже існує в курсі", content = @Content(schema =
@Schema(implementation = ApiErrorResponse.class))),
    @ApiResponse(responseCode = "500", description = "Внутрішня помилка
сервера", content = @Content(schema = @Schema(implementation =
ApiErrorResponse.class)))
})
@PostMapping("/modules")
public ResponseEntity<ModuleResponse> create(@Valid @RequestBody
ModuleCreateRequest request) {
    return ResponseEntity.ok(toResponse(service.create(request)));
}

@Operation(summary = "Отримати всі модулі", description = "Повертає список
усіх навчальних модулів.")
@ApiResponses({
    @ApiResponse(responseCode = "200", description = "Список модулів
отримано", content = @Content(array = @ArraySchema(schema = @Schema(implementation
= ModuleResponse.class)))),
    @ApiResponse(responseCode = "500", description = "Внутрішня помилка
сервера", content = @Content(schema = @Schema(implementation =
ApiErrorResponse.class)))
})
@GetMapping("/modules")
public ResponseEntity<List<ModuleResponse>> getAll() {
    return
ResponseEntity.ok(service.getAll().stream().map(this::toResponse).toList());
}

@Operation(summary = "Отримати модуль за id", description = "Повертає
навчальний модуль за ідентифікатором.")
@ApiResponses({
    @ApiResponse(responseCode = "200", description = "Модуль знайдено",
content = @Content(schema = @Schema(implementation = ModuleResponse.class))),
    @ApiResponse(responseCode = "404", description = "Модуль не знайдено",
content = @Content(schema = @Schema(implementation = ApiErrorResponse.class))),
    @ApiResponse(responseCode = "500", description = "Внутрішня помилка
сервера", content = @Content(schema = @Schema(implementation =
ApiErrorResponse.class)))
})
@GetMapping("/modules/{id}")
public ResponseEntity<ModuleResponse> getById(@PathVariable Long id) {
    return ResponseEntity.ok(toResponse(service.getById(id)));
}

@Operation(summary = "Оновити модуль", description = "Оновлює дані навчального
модуля. Перевіряється валідність DTO, існування курсу та унікальність порядкового
номера.")

```

```

    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Модуль оновлено",
            content = @Content(schema = @Schema(implementation = ModuleResponse.class))),
        @ApiResponse(responseCode = "400", description = "Некоректні дані модуля або помилки валідації", content = @Content(schema = @Schema(implementation =
            ApiErrorResponse.class))),
        @ApiResponse(responseCode = "404", description = "Модуль або курс не знайдено", content = @Content(schema = @Schema(implementation =
            ApiErrorResponse.class))),
        @ApiResponse(responseCode = "409", description = "Модуль з таким порядковим номером вже існує в курсі", content = @Content(schema =
            @Schema(implementation = ApiErrorResponse.class))),
        @ApiResponse(responseCode = "500", description = "Внутрішня помилка сервера", content = @Content(schema = @Schema(implementation =
            ApiErrorResponse.class)))
    })
    @PutMapping("/modules/{id}")
    public ResponseEntity<ModuleResponse> update(
        @PathVariable Long id,
        @Valid @RequestBody ModuleUpdateRequest request) {
        return ResponseEntity.ok(toResponse(service.update(id, request)));
    }

    @Operation(summary = "Видалити модуль", description = "Видаляє навчальний модуль за ідентифікатором.")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Модуль видалено",
            content = @Content(schema = @Schema(implementation = String.class))),
        @ApiResponse(responseCode = "404", description = "Модуль не знайдено",
            content = @Content(schema = @Schema(implementation = ApiErrorResponse.class))),
        @ApiResponse(responseCode = "500", description = "Внутрішня помилка сервера", content = @Content(schema = @Schema(implementation =
            ApiErrorResponse.class)))
    })
    @DeleteMapping("/modules/{id}")
    public ResponseEntity<String> delete(@PathVariable Long id) {
        service.delete(id);
        return ResponseEntity.ok("Deleted module " + id);
    }

    @Operation(summary = "Додати модуль до курсу", description = "Прив'язує існуючий модуль до курсу за ідентифікаторами курсу та модуля.")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Модуль додано до курсу", content = @Content(schema = @Schema(implementation =
            ModuleResponse.class))),
        @ApiResponse(responseCode = "404", description = "Курс або модуль не знайдено", content = @Content(schema = @Schema(implementation =
            ApiErrorResponse.class))),
        @ApiResponse(responseCode = "409", description = "У курсі вже є модуль з таким порядковим номером", content = @Content(schema = @Schema(implementation =
            ApiErrorResponse.class))),
        @ApiResponse(responseCode = "500", description = "Внутрішня помилка сервера", content = @Content(schema = @Schema(implementation =
            ApiErrorResponse.class)))
    })
    @PostMapping("/courses/{id}/modules/{moduleId}")
    public ResponseEntity<ModuleResponse> addModule(@PathVariable Long id,
        @PathVariable Long moduleId) {
        return ResponseEntity.ok(toResponse(service.addToCourse(id, moduleId)));
    }
}

```

```

        @Operation(summary = "Видалити модуль з курсу", description = "Видаляє зв'язок між модулем і курсом без видалення самого модуля.")
        @ApiResponses({
            @ApiResponse(responseCode = "200", description = "Модуль видалено з курсу", content = @Content(schema = @Schema(implementation = String.class))),
            @ApiResponse(responseCode = "404", description = "Курс, модуль або зв'язок модуля з курсом не знайдено", content = @Content(schema = @Schema(implementation = ApiErrorResponse.class))),
            @ApiResponse(responseCode = "500", description = "Внутрішня помилка сервера", content = @Content(schema = @Schema(implementation = ApiErrorResponse.class)))
        })
        @DeleteMapping("/courses/{id}/modules/{moduleId}")
        public ResponseEntity<String> removeModule(@PathVariable Long id, @PathVariable Long moduleId) {
            service.removeFromCourse(id, moduleId);
            return ResponseEntity.ok("Removed module " + moduleId + " from course " + id);
        }

        private ModuleResponse toResponse(CourseModule module) {
            return new ModuleResponse(
                module.getId(),
                module.getTitle(),
                module.getDescription(),
                module.getOrderIndex(),
                module.getCourse() == null ? null : module.getCourse().getId()
            );
        }
    }
}

```

Лістинг Д.6 – Код класу TextSearchController

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import ua.opnu.labwork2.course.dto.CourseResponse;
import ua.opnu.labwork2.course.model.Course;
import ua.opnu.labwork2.course.repository.CourseRepository;
import ua.opnu.labwork2.student.dto.StudentResponse;
import ua.opnu.labwork2.student.model.Student;
import ua.opnu.labwork2.student.repository.StudentRepository;

import java.util.List;

@RestController
@RequestMapping(TextSearchController.BASE_PATH)
public class TextSearchController {
    public static final String BASE_PATH = "/search";

    @Autowired
    private CourseRepository courseRepository;

    @Autowired
    private StudentRepository studentRepository;

```

```

    @GetMapping("/courses")
    public ResponseEntity<List<CourseResponse>> searchCourses(@RequestParam String
query) {
        return ResponseEntity.ok(courseRepository

.findByTitleContainingIgnoreCaseOrDescriptionContainingIgnoreCase(query, query)
            .stream()
            .map(this::toCourseResponse)
            .toList());
    }

    @GetMapping("/courses/advanced")
    public ResponseEntity<List<CourseResponse>> advanced(@RequestParam String
query, @RequestParam String level) {
        return ResponseEntity.ok(courseRepository.searchByQueryAndLevel(query,
level)
            .stream()
            .map(this::toCourseResponse)
            .toList());
    }

    @GetMapping("/students")
    public ResponseEntity<List<StudentResponse>> searchStudents(@RequestParam
String query) {
        return ResponseEntity.ok(studentRepository

.findByFirstNameContainingIgnoreCaseOrLastNameContainingIgnoreCaseOrEmailContainin
gIgnoreCase(
            query,
            query,
            query
        )
            .stream()
            .map(this::toStudentResponse)
            .toList());
    }

    private CourseResponse toCourseResponse(Course course) {
        return new CourseResponse(
            course.getId(),
            course.getTitle(),
            course.getDescription(),
            course.getLevel(),
            course.getCreatedDate(),
            course.getInstructor() == null ? null :
course.getInstructor().getId()
        );
    }

    private StudentResponse toStudentResponse(Student student) {
        return new StudentResponse(
            student.getId(),
            student.getFirstName(),
            student.getLastName(),
            student.getEmail(),
            student.getRegistrationDate()
        );
    }
}

```

Лістинг Д.7 – Код класу AnalyticsController

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import ua.opnu.labwork2.course.repository.CourseRepository;
import ua.opnu.labwork2.enrollment.repository.EnrollmentRepository;
import ua.opnu.labwork2.student.repository.StudentRepository;

import java.util.LinkedHashMap;
import java.util.Map;

@RestController
@RequestMapping(AnalyticsController.BASE_PATH)
public class AnalyticsController {
    public static final String BASE_PATH = "/analytics";

    @Autowired
    private CourseRepository courseRepository;

    @Autowired
    private StudentRepository studentRepository;

    @Autowired
    private EnrollmentRepository enrollmentRepository;

    @GetMapping("/courses/count")
    public ResponseEntity<Long> coursesCount() {
        return ResponseEntity.ok(courseRepository.count());
    }

    @GetMapping("/students/count")
    public ResponseEntity<Long> studentsCount() {
        return ResponseEntity.ok(studentRepository.count());
    }

    @GetMapping("/enrollments/active")
    public ResponseEntity<Long> active() {
        return
ResponseEntity.ok(enrollmentRepository.countByCompletionDateIsNull());
    }

    @GetMapping("/enrollments/completed")
    public ResponseEntity<Long> completed() {
        return
ResponseEntity.ok(enrollmentRepository.countByCompletionDateIsNotNull());
    }

    @GetMapping("/courses/by-level")
    public ResponseEntity<Map<String, Long>> byLevel() {
        Map<String, Long> result = new LinkedHashMap<>();
        for (Object[] row : courseRepository.countCoursesByLevel()) {
            String level = row[0] == null ? "UNKNOWN" : row[0].toString();
            Long count = (Long) row[1];
            result.put(level, count);
        }
        return ResponseEntity.ok(result);
    }
}

```

Лістинг Д.8 – Код класу ActuatorController

```

import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping(ActuatorController.BASE_PATH)
public class ActuatorController {
    public static final String BASE_PATH = "/actuator";

    @GetMapping("/health")
    public ResponseEntity<Map<String, Object>> health() {
        return ResponseEntity.ok(Map.of(
            "status", "UP",
            "service", "labwork-2",
            "timestamp", LocalDateTime.now()
        ));
    }

    @GetMapping("/metrics")
    public ResponseEntity<Map<String, Object>> metrics() {
        return ResponseEntity.ok(Map.of(
            "availableMetrics", List.of(
                "courses.count",
                "students.count",
                "enrollments.active",
                "enrollments.completed",
                "courses.byLevel"
            )
        ));
    }

    @GetMapping(value = "/prometheus", produces = MediaType.TEXT_PLAIN_VALUE)
    public ResponseEntity<String> prometheus() {
        String metrics = ""
            # HELP application_up Application health status
            # TYPE application_up gauge
            application_up 1
            # HELP application_info Application information
            # TYPE application_info gauge
            application_info{service="labwork-2"} 1
            "";
        return ResponseEntity.ok(metrics);
    }
}

```

ДОДАТОК Е

Конфігураційні файли проєкту

У додатку Е наведено конфігураційні класи, файли налаштування та класи для централізованої обробки помилок. Вони забезпечують документування API, підключення бази даних та формування єдиного формату відповіді у випадку помилки.

Лістинг Е.1 – Код класу OpenApiConfig

```
import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Contact;
import io.swagger.v3.oas.models.info.Info;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI onlineCoursesOpenApi() {
        return new OpenAPI()
            .info(new Info()
                .title("Система управління онлайн-курсами")
                .version("1.0.0")
                .description("""
                    Backend API для управління онлайн-курсами,
                    викладачами, студентами,
                    навчальними модулями та записами студентів на
                    курси. Система реалізована
                    на Spring Boot, Spring MVC, Spring Data JPA,
                    Jakarta Bean Validation та PostgreSQL.
                    API використовує DTO для вхідних і вихідних даних,
                    централізовану обробку помилок
                    та власні класи винятків.
                    """))
            .contact(new Contact()
                .name("Кафедра комп'ютерних систем")
                .email("antonova.katya2006@gmail.com"));
    }
}
```

Лістинг Е.2 – Код класу ApiErrorResponse

```
import io.swagger.v3.oas.annotations.media.Schema;

import java.time.LocalDateTime;
import java.util.Map;

@Schema(description = "Єдиний формат відповіді сервера у випадку помилки")
```

```

public record ApiErrorResponse(
    @Schema(description = "Час виникнення помилки на сервері", example =
"2026-05-10T18:30:00")
    LocalDateTime timestamp,

    @Schema(description = "HTTP-статус помилки", example = "400")
    int status,

    @Schema(description = "Назва HTTP-помилки", example = "Bad Request")
    String error,

    @Schema(description = "Загальне повідомлення про помилку", example =
"Validation failed")
    String message,

    @Schema(description = "URL запиту, під час якого виникла помилка", example
= "/courses")
    String path,

    @Schema(description = "Помилки валідації окремих полів", example =
"{\"title\": \"Course title is required\"}")
    Map<String, String> errors
) {
}

```

Лістинг Е.3 – Код класу GlobalExceptionHandler

```

import jakarta.servlet.http.HttpServletRequest;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.http.converter.HttpMessageNotReadableException;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.method.annotation.MethodArgumentTypeMismatchException;

import java.time.LocalDateTime;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.NoSuchElementException;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiErrorResponse> handleValidation(
        MethodArgumentNotValidException ex,
        HttpServletRequest request) {
        Map<String, String> errors = new LinkedHashMap<>();
        for (FieldError fieldError : ex.getBindingResult().getFieldErrors()) {
            errors.put(fieldError.getField(), fieldError.getDefaultMessage());
        }
        return build(HttpStatus.BAD_REQUEST, "Validation failed",
request.getRequestURI(), errors);
    }
}

```

```

    @ExceptionHandler({
        BadRequestException.class,
        MethodArgumentTypeMismatchException.class,
        HttpResponseMessageNotReadableException.class
    })
    public ResponseEntity<ApiErrorResponse> handleBadRequest(Exception ex,
HttpServletRequest request) {
        return build(HttpStatus.BAD_REQUEST, ex.getMessage(),
request.getRequestURI(), null);
    }

    @ExceptionHandler({ResourceNotFoundException.class,
NoSuchElementException.class})
    public ResponseEntity<ApiErrorResponse> handleNotFound(Exception ex,
HttpServletRequest request) {
        return build(HttpStatus.NOT_FOUND, ex.getMessage(),
request.getRequestURI(), null);
    }

    @ExceptionHandler({
        DuplicateResourceException.class,
        CourseEnrollmentException.class,
        ModuleOrderException.class,
        DataIntegrityViolationException.class
    })
    public ResponseEntity<ApiErrorResponse> handleConflict(Exception ex,
HttpServletRequest request) {
        return build(HttpStatus.CONFLICT, ex.getMessage(),
request.getRequestURI(), null);
    }

    @ExceptionHandler({DatabaseOperationException.class, Exception.class})
    public ResponseEntity<ApiErrorResponse> handleServerError(Exception ex,
HttpServletRequest request) {
        return build(
            HttpStatus.INTERNAL_SERVER_ERROR,
            "Internal server error",
            request.getRequestURI(),
            null
        );
    }

    private ResponseEntity<ApiErrorResponse> build(
        HttpStatus status,
        String message,
        String path,
        Map<String, String> errors) {
        ApiErrorResponse response = new ApiErrorResponse(
            LocalDateTime.now(),
            status.value(),
            status.getReasonPhrase(),
            message,
            path,
            errors
        );
        return ResponseEntity.status(status).body(response);
    }
}

```

```
public class BadRequestException extends RuntimeException {  
    public BadRequestException(String message) {  
        super(message);  
    }  
}
```

Лістинг E.5 – Код класу ResourceNotFoundException

```
public class ResourceNotFoundException extends RuntimeException {  
    public ResourceNotFoundException(String message) {  
        super(message);  
    }  
  
    public ResourceNotFoundException(String resourceName, Long id) {  
        super(resourceName + " with id " + id + " was not found");  
    }  
}
```

Лістинг E.6 – Код класу DuplicateResourceException

```
public class DuplicateResourceException extends RuntimeException {  
    public DuplicateResourceException(String message) {  
        super(message);  
    }  
}
```

Лістинг E.7 – Код класу CourseEnrollmentException

```
public class CourseEnrollmentException extends RuntimeException {  
    public CourseEnrollmentException(String message) {  
        super(message);  
    }  
}
```

Лістинг E.8 – Код класу ModuleOrderException

```
public class ModuleOrderException extends RuntimeException {  
    public ModuleOrderException(String message) {  
        super(message);  
    }  
}
```

Лістинг E.9 – Код класу DatabaseOperationException

```
public class DatabaseOperationException extends RuntimeException {  
    public DatabaseOperationException(String message) {  
        super(message);  
    }  
}
```

```
    }  
}
```

Лістинг Е.10 – Файл налаштувань application.properties

```
spring.application.name=labwork-2  
  
spring.datasource.url=jdbc:postgresql://localhost:5432/university_db  
spring.datasource.username=user_db  
spring.datasource.password=useruser  
spring.datasource.driver-class-name=org.postgresql.Driver  
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect  
spring.jpa.hibernate.ddl-auto=update
```
